

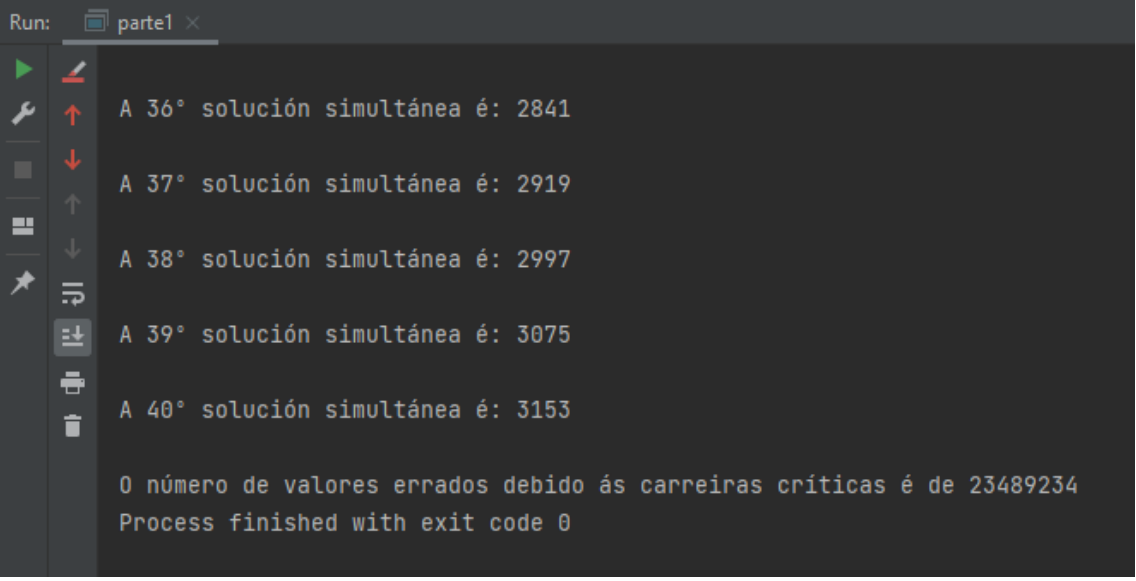
BITÁCORAS SISTEMAS OPERATIVOS II

2020-2021

2º curso – 2º cuatrimestre

Os seguintes apuntamentos foron recollidos en forma de bitácoras durante cada clase do curso polas seguintes persoas:

Adriana Aurora Rodríguez Oreiro, Adrián Vidal Lorenzo, Andrea Solla Alfonsín, Andrés Balea Domínguez, Belén Valle Cao, Diana Mascareñas Sande, Elena Segade Martínez, Hugo Vázquez Docampo, Inés Quintana Raña, Javier Beiro Piñón, Manuel Cid Domínguez, Martín Campos Zamora, Miguel Méndez Martínez, Nerea Freiría Alonso, Nicolás Vilela Pérez, Pablo Gil Pérez, Pablo Martínez González, Roberto de la Iglesia, Roque Fernández Iglesias, Teresa Gutiérrez Blanco



```
Run: parte1 x
A 36° solución simultánea é: 2841
A 37° solución simultánea é: 2919
A 38° solución simultánea é: 2997
A 39° solución simultánea é: 3075
A 40° solución simultánea é: 3153

0 número de valores errados debido ás carreiras críticas é de 23489234
Process finished with exit code 0
```

Non son apuntamentos recollidos de forma 100% formal e pode haber erros nalgúns datos ou fallos de formato. A pesar disto, pensamos que poden ser de gran axuda para os vindeiros cursos :)



Escola
Técnica
Superior
de Enxeñaría



Grado en Ingeniería Informática

2º CURSO – 2º CUATRIMESTRE

SISTEMAS OPERATIVOS II

11-02-2021

Autores:



Clase 00

Temas trabajados:
Presentación

Índice

1. Horario	2
1.1 Clases expositivas:	2
1.2 Clases interactivas:	2
2. Temario	2
3. Sistema de evaluación	2
4. Bibliografía	3
5. Repaso de Conceptos Básicos	3
6. EJERCICIO DE REPASO :D	4

1. Horario

1.1 Clases expositivas:

- Jueves de 16:30 a 17:30

1.2 Clases interactivas:

- Grupo 1: Jueves de 9:30 a 12:00. (Aula I5 y Teams)
- Grupo 2: Martes de 10:00 a 12:30. (Aula I3 y Teams)
- Grupo 3: Viernes de 11:30 a 14:00. (Aula I5 y Teams)
- Grupo 4: Lunes de 9:00 a 11:30. (Aula I5 y Teams)

2. Temario

- 1. Comunicación y sincronización entre procesos e hilos.
- 2. Interbloqueos.
- 3. Sistemas operativos distribuidos.
- 4. Sistemas operativos en tiempo real.
- 5. Programación del shell.
 - Creación de scripts (solo se explicarán en prácticas).

Importante:

- Los temas del 1 al 4 tendrán **2 partes** una interactiva y otra expositiva. El tema 5, se explicará **únicamente** en práctica aún así **SIEMPRE** cae una pregunta en el examen final teórico
- En cada tema aparecerán recuadros en la parte inferior izquierda con la información de los capítulos y las secciones del Tannenbaum que explican el contenido dado.

3. Sistema de evaluación

- **65 % teoría:** Examen.
- **35 % practicas:** Asistencia, participación, Evaluación Continua.
- **+20 % adicional:** ejercicios.
 - Se pondrán calificaciones a los ejercicios que se van haciendo para poder calcular la nota que se tendrá en esta parte a medida que avanza el cuatrimestre.

Importante: Ambas partes se **tienen** que aprobar independientemente. Sin embargo, si alguna de las dos está aprobada de forma holgada, se puede aprobar una de ellas con un suspenso. (Lo dijo con un poco de reticencia, seguramente no pueda ser menos de y 4, 4.5)

4. Bibliografía

- Andrew S. **Tanenbaum** Sistema operativos modernos (3ª edición) Editorial Prentice-Hall, 2009
 - [Descargar aquí](#)
- J. Carrertero, F.García, P. de Miguel e F.Pérez. Sistemas Operativas: una visión aplicada. (2ª edición) McGraw-Hill, 2007.
 - [Descargar aquí](#)

Descarga el **Tanenbaum** por tu bien :/

5. Repaso de Conceptos Básicos

- Sistema Operativo
- Procesos e hilos
 - **Multiprogramación**
 - Estado de los procesos
 - Repaso de los 3 estados básicos de un proceso. Bloqueado, listo y en ejecución.
 - Tabla de procesos
 - Hilos(Compartición del espacio de direcciones)
 - Planificación: procesos - hilos
 - POSIX: fork y execv
 - Win32: create_process
- Planificación
- Interrupciones
- Llamadas al sistema
- Entrada/Salida
- Gestión de la memoria virtual
- El shell

Información:

- Todos estos conceptos se pueden encontrar en los Temas 1 y 2 del Tanenbaum.
- A lo largo de las clases expositivas se utilizarán los términos hilo y procesos como sinónimos.

6. EJERCICIO DE REPASO :D

¿Cuál es la salida de este código?

```
#include <stdio.h>
#include <unistd.h>

int main(){
    int i;
    for(i=0;i<3;i++){
        fork();
        printf("i=%d\n",i);
    }
}
```

Solución:

```
i=0
i=0
i=1
i=1
i=1
i=1
i=2
i=2
i=2
i=2
i=2
i=2
i=2
i=2
```



Escola
Técnica
Superior
de Enxeñaría

Grado en Ingeniería Informática 2º curso – 2º cuatrimestre

Sistemas Operativos II - 18-02-2021



Clase 02

- Canción del día: The Bangles - Hazy Shade of Winter

<https://www.youtube.com/watch?v=TxrwImCJCgk>

TEMA 1 - Comunicación y sincronización entre procesos e hilos

```
#include <stdio.h>
#include <unistd.h>
main() {
    int i;
    for (i=0;i<3;i++) {
        fork();
        printf("i=%d\n",i); }
}
```

1. ¿Cuántos procesos están involucrados en el código cuando se ejecuta?

Cuando se ejecuta el código, se crearán finalmente 8 procesos en total siguiendo el siguiente esquema:

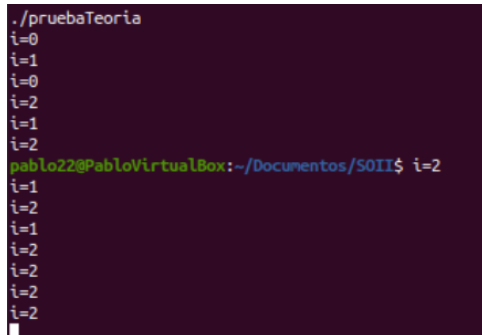
- En la primera vuelta del lazo se tiene el proceso Padre, y se crea mediante fork() el Hijo1, teniendo así dos procesos, Padre e Hijo1.
- En la segunda interacción se va a crear un hijo de cada proceso que se tenía hasta entonces, de manera que el Padre creará Hijo2, y el proceso Hijo1 creará Nieto1, acabando con 4 procesos: Padre, Hijo1, Hijo2 y Nieto1.
- En la última interacción, el Padre creará Hijo3, Hijo1 creará Nieto2, Hijo2 creará Nieto 3, y Nieto1 creará Bisnieto1. De esta manera en la ejecución estarán los procesos: Padre, Hijo1, Hijo2, Nieto1, Hijo3, Nieto2, Nieto3 y Bisnieto, 8 en total.

2. ¿Qué vemos en la salida de la consola?

La salida de la ejecución será la que se muestra en la figura, mostrándose el mensaje de la función printf() una vez por cada proceso. Consecuentemente, se mostrará el numero dependiendo de en qué interacción del bucle se haya creado dicho proceso, teniendo así dos i=0, cuatro veces i=1, y ocho veces i=2. La razón de que aparezcan desordenadas se debe a que los procesos no esperan por otros para continuar, por lo que cada proceso puede encontrarse en una interacción distinta del lazo que los demás. Un ejemplo de ello serían las tres primeras impresiones, que son i=0, i=1, i=0, donde uno de los procesos del primer lazo (Padre e Hijo1) imprime i=0 y pasa a la siguiente interacción del lazo, imprimiendo i=1 sin esperar al otro.

3. ¿En qué momento vemos el prompt?

Como se puede ver en la imagen de salida del código, el prompt aparece antes de que se terminen de imprimir todos los mensajes. Esto es consecuencia de lo comentado anteriormente, ya que el proceso padre no espera por los demás para terminar, si no que finaliza, haciendo que se muestre el prompt por pantalla, a pesar de que los demás procesos creados anteriormente prosigan con su ejecución, imprimiendo sus respectivos mensajes.



```
./pruebaTeoria
i=0
i=1
i=0
i=2
i=1
i=2
pablo22@PabloVirtualBox:~/Documentos/SOII$ i=2
i=1
i=2
i=1
i=2
i=2
i=2
i=2
```

Comunicación entre procesos (IPC)

En ocasiones, es necesaria la comunicación entre procesos diferentes para poder llevar a cabo su ejecución. Un ejemplo de comunicación entre procesos es el **pipe** (canalización en el shell). Cuando se escribe “un comando | otro comando”, la salida del primer proceso, correspondiente al primer comando, se almacena en el pipe, que es un fichero temporal que se va a utilizar como entrada del segundo proceso.

Las señales sirven para comunicarse sin mover datos o información, únicamente enviando avisos de un proceso a otro

La comunicación entre procesos se usa para:

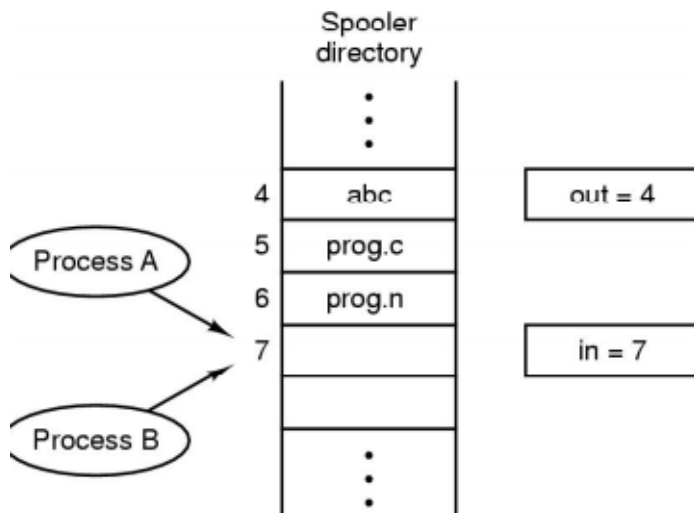
1. Pasar información entre procesos.
2. Evitar interferencias: ya que, si un proceso está realizando una tarea, otro proceso puede interferir, afectando a su buen funcionamiento o produciendo carreras críticas.
3. Garantizar un orden correcto (Sincronización): Este tipo de situaciones están asociadas a las dependencias. Un proceso puede depender del resultado o de la ejecución de otro, como esperar a que otro proceso termine.

Los dos últimos puntos se pueden tratar mediante las mismas herramientas tanto para hilos como procesos. Sin embargo, el primer punto es más sencillo para los hilos ya que comparten el espacio de direcciones, por lo que la transferencia de información se hace de manera directa. Sin embargo, entre procesos no se comparte memoria salvo que explícitamente compartan espacio virtual de memoria, o enviando un mensaje entre procesos (funciones de envío y recepción de mensajes).

El principal inconveniente es que los sistemas son **multiprogramados**, es decir, varios hilos y procesos ejecutándose al mismo tiempo, por lo que pueden interferir entre sí. Además, el efecto de la multiprogramación no se puede prever, por lo que no se tiene garantías de que el orden de ejecución sea el mismo.

Condiciones de carrera (condiciones de competencia)

Este es uno de los puntos más importantes a tratar. Ejemplo: spooler de impresión



La imagen nos muestra el **buffer de impresión**. Todos los procesos que hay en el sistema, cuando lanzan una actividad de impresión, escriben en la tabla el nombre del fichero que se pretende imprimir. Todos los procesos tienen acceso a esa tabla. La labor que tienen que hacer los procesos es decir lo que quieren imprimir, y escribir el nombre del fichero en el buffer, que se sitúa en el **Kernel** ya que es una zona compartida por todos los procesos.

Otro aspecto importante es el **demonio de impresión**, que es el encargado de ir al buffer, ver si hay algo que imprimir, y si lo hay coger la primera posición, leer el fichero y hacer el trabajo correspondiente, comunicándose con la impresora.

La variable "out", tiene la posición de la cola donde se encuentra el archivo que lleva más tiempo esperando para ser impreso. Cuando el demonio de impresión realice el trabajo, lee el valor de la variable y realiza las correspondientes operaciones para imprimir lo correspondiente.

La variable "in" indica la posición del buffer en la que hay que añadir un nuevo fichero, es decir, es la siguiente posición libre de la cola. Esta variable la usan, los procesos para escribir en el buffer nuevos ficheros.

Podemos definir este sistema como una cola **FIFO (First In, First Out)**. Sin embargo, es un sistema no válido debido a que puede producir errores, explicados en las siguientes sesiones. Según la ley de Murphy, esta técnica FIFO es incorrecta.

- LEY DE MURPHY: Si existe una posibilidad de que un código, método o estrategia esté mal, por muy remota que sea, es que se ha programado mal, por lo que está mal hecho.



Escola
Técnica
Superior
de Enxeñaría



Grado en Ingeniería Informática

2º curso – 2º cuatrimestre

Sistemas Operativos 2

25-02-2021



Clase 3

Temas trabajados:

Carreras Críticas

Índice

1. Carreras Críticas	3
Introducción	3
Ejemplo 1.....	3
Ejemplo 2.....	5

1. Carreras Críticas

Introducción

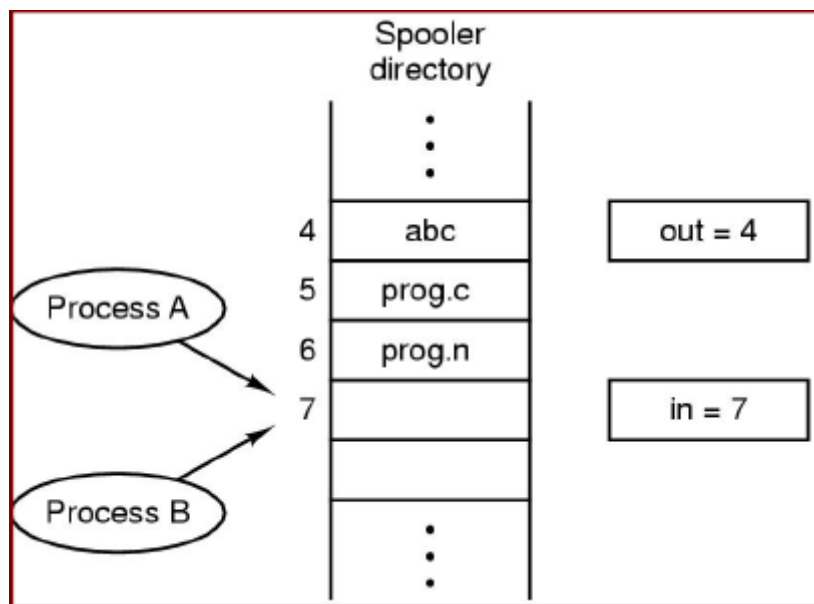
Comienza la clase con el típico temón de Rivera, hoy se trata de Born to Be Wild, movemos un poco el esqueleto y comienza la sesión. Comienza repasando ligeramente el final de la clase anterior, sobre condiciones críticas y se pone a explicar más sobre estas carreras críticas, la clase se desarrolla en las diapositivas 3 y 4 de la presentación del tema 1 de la materia.

Las carreras críticas son problemas que se producen en situaciones de competencia y que afectan al funcionamiento del SO. Es un problema difícil de detectar y de solucionar por lo que la solución de dicho problema corresponde al programador. El SO no nos proporciona asistencia para hacer frente a este problema ya que es de los que más lo sufre. Puede ocurrir en los procesos del propio SO y producir errores que conlleven a pantallazos azules.

Podríamos definir las carreras críticas como ejecuciones de un programa erróneas debido a un cambio de contexto o una interrupción de reloj en un momento crítico de la ejecución. Es decir, que un proceso pierda la CPU en un momento importante y continúe otro que acaba afectando al que se ejecutaba primero.

Ejemplo 1

Para explicar esto se ayudó de este ejemplo:



En el ejemplo vemos una figura de una cola FIFO de impresión compartida por dos procesos (A y B). El puntero in representa la primera posición vacía, mientras que el puntero out representa

la 1ª posición a leer para imprimir. Hay otro proceso llamado demonio de impresión que se encarga de la gestión de la cola. El demonio comprueba el puntero out y si lleva a una posición no vacía entonces imprime el contenido de dicha posición. Si out = in significa que no hay nada para imprimir en la cola. Los procesos A y B introducen ficheros en la cola mirando el puntero in. Es en la actualización del puntero en donde se podrían producir carreras críticas. Imaginémonos que el código que realizan los procesos es el siguiente.

```
//proceso A
```

```
copy (in,fichero);
in = in + 1;
```

```
//proceso B
```

```
copy (in,codigo);
in = in + 1;
```

Este código puede fallar si A ejecuta la instrucción copy y se produce un cambio de contexto y se entrega el uso de la CPU al proceso B. El proceso B ejecutará también su instrucción copy, por lo que sobrescribirá el contenido que escribió A y ejecutará la reasignación del puntero. Cuando A recupere la CPU también reasignará el puntero, por lo que al final tendremos la variable de A machacada por la de B y una posición vacía.

Podríamos pensar que se podría solucionar cambiando el código a:

```
//proceso A
```

```
in = in + 1;
copy (in - 1,fichero);
```

```
//proceso B
```

```
in = in + 1;
copy (in - 1,codigo);
```

Pero tampoco lo solucionaría porque seguimos haciendo un acceso a una variable compartida. Para poder arreglarlo tendremos que pensar cómo sería la secuencia de instrucciones en lenguaje ensamblador, que sería aproximadamente de la siguiente forma:

```
lw R1, in
addi R1, R1, 1
sw R1, in
.
.
.
jal copy
```

Tendemos a pensar que una sentencia de alto nivel se ejecuta de una sola tacada, pero esto no es así ya que puede haber una interrupción en medio de la sentencia, si dicha sentencia está

compuesta por varias instrucciones ensamblador (como suele ocurrir). La granularidad de las instrucciones es mucho más fina de lo que pensamos. En el ejemplo, la raíz del problema se produce por el uso compartido de la variable `in`. Las variables compartidas pueden producir carreras críticas si varios procesos las intentan actualizar simultáneamente. El proceso demonio también podría estar implicado en carreras críticas dependiendo de su implementación.

Al usar varios hilos hay que tener cuidado con las variables globales para evitar carreras críticas. Con procesos no hay este problema, ya que a priori no comparten memoria, aunque haya formas de que sí que lo hagan. Todas las variables del kernel son susceptibles de sufrir carreras críticas ya que podrían ser accedidas por cualquier proceso en ejecución en modo núcleo. Las carreras críticas son poco frecuentes pero posibles, además de resultar impredecibles en la inmensa mayoría de casos.

Ejemplo 2

Otro ejemplo que pone es el que se puede ver en la diapositiva 4 del tema. La primera pieza de código muestra una manera de calcular una suma parcial mediante hilos, teniendo la variable suma compartida por todos los hilos del programa y modificando $n/4$ veces por cada hilo. Al ser una variable compartida, como vimos anteriormente puede producir una carrera crítica en cada iteración del for, ya que todos los hilos intentan leer y escribir.

Se propone hacer en clase un script que ejecute un programa que use los hilos de esa manera para hacer la suma secuencial, y que lo hagamos un millón de veces y veamos cuando se produce una carrera crítica, que será cuando la suma de diferente a lo que debería darnos.

La otra solución que da se trata de un código con una variable local, se calcula en ella la suma parcial del hilo y por último se suma esa variable local a la suma total. Se propone hacer un video para el flipgrid explicando si se pudieran producir carreras críticas ahora que hemos cambiado el código y hemos añadido esta variable local.

La respuesta a lo anterior es SI, se sigue escribiendo sobre una variable compartida por lo que podría producirse en algún momento de su ejecución. En C no podremos ver donde puede ocurrir, pero en ensamblador vemos que se producen 4 instrucciones, 2 `lw`, un `add` y el `sw` final. Si tras hacer el `lw` de suma sobre un registro y pasar a los siguientes pasos el proceso 1 pierde la CPU y otro proceso 2 llega a sumar tendremos que el valor de suma ha sido modificado y el proceso 1 anterior no lo detectará, y sobrescribirá el valor correcto de la suma, desperdiciando lo sumado por el proceso 2.

Tras subir los videos, el profe se despide con un leve “hasta luego”, parece afectado por algo pero nadie le pregunta, seguramente haberle pegado demasiado al perro le pase factura, termina la clase.



Escola
Técnica
Superior
de Enxeñaría



Grado en Ingeniería Informática

2º curso – 2º cuatrimestre

Sistemas Operativos II

04-03-2021

Clase 04

Temas trabajados:

Regiones Críticas y posibles soluciones

Índice

1. Carreras Críticas	3
Repaso de la clase anterior	3
2. Región Crítica	4
3. Posibles Soluciones para las carreras críticas	6
EXCLUSIÓN MUTUA CON ESPERA OCUPADA.....	6
VARIABLES DE CANDADO (O BLOQUEO).....	6
ALTERNANCIA ESTRICTA	6

1. Carreras Críticas

Repaso de la clase anterior

```
int suma=0;
...
void suma(int ni, int nf)
{
    int j;
    for (j=ni; j<=nf; j++)
        suma = suma + j; }
```

Hay una variable que es suma que se ejecuta en todos los hilos que formen parte del proceso ya que es una variable compartida, es decir, es global. En la siguiente línea de código:

suma = suma+ j

Se ejecutan las instrucciones en código pseudo mips:

1- lw R1 j

2- lw R2 suma

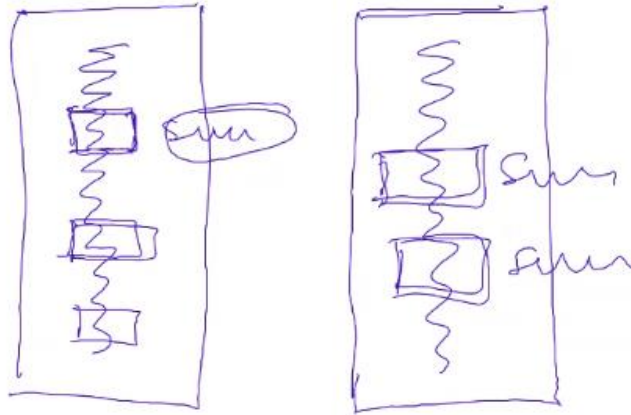
3- add R2 R2 R1

4- sw R2, suma

En cualquiera de las instrucciones anteriores, se puede producir un cambio de contexto. Sin embargo, si se producen cambios de contexto entre las instrucciones 2 y 3, o entre las 3 y 4, se pueden producir carreras críticas. Para ello, además se tiene que dar la CPU a otro hilo que también pretenda acceder a la variable suma, ya que de esta forma podría modificar el contenido de la variable antes de que el primer hilo ejecutase su instrucción, provocando resultados incorrectos.

2. Región Crítica

Parte del programa que accede a memoria compartida que puede dar lugar a carreras críticas. Se necesita de exclusión mutua, que se define más tarde.



Durante la ejecución de ambos hilos, en el código de estos se acceden a variables compartidas, como es el caso de la variable suma. Durante este acceso, el proceso se encuentra dentro de una de las regiones críticas del programa.

Interesa que la región crítica sea lo más pequeña posible, incluso en términos de lenguaje máquina. Por ejemplo:

Suma = suma+ j. En esta instrucción la región crítica se encuentra en los puntos 2,3 y 4

2- lw R2 suma

3- add R2, R2, R1

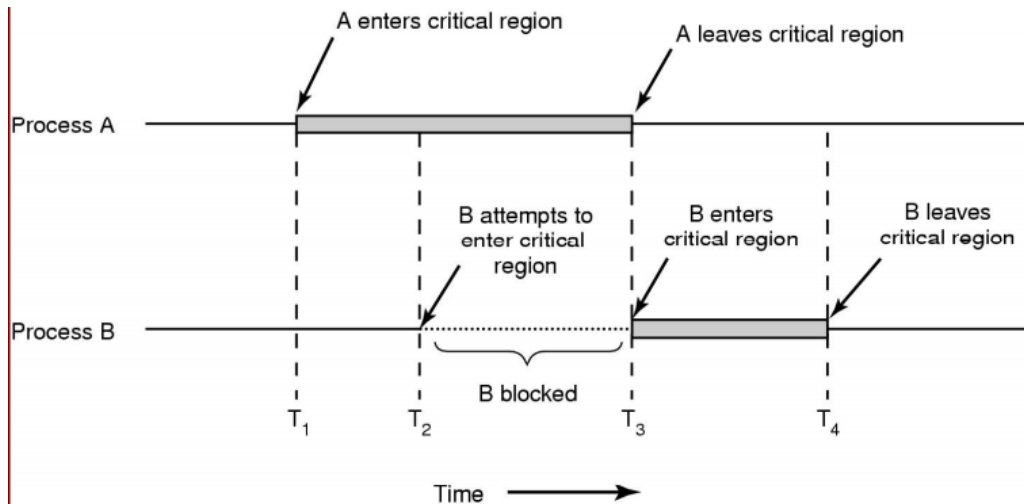
4- sw R2, suma

¿Cómo se identifican las regiones críticas?

Para poder identificar las regiones críticas se debe tener plena conciencia del funcionamiento del programa, y además ser capaces de imaginarse las instrucciones en ensamblador. Por otro lado, como las regiones críticas se producen cuando varios programas acceden a memoria compartida, se debe conocer los códigos de todos aquellos programas que vayan a acceder a esas variables comunes. Además, hay que tener en cuenta que hay algunas variables que comparte todo el sistema operativo, por lo que se el control de las regiones críticas debe ser mayor.

¿Cómo se solucionan las regiones críticas?

Garantizando **exclusión mutua**. La exclusión mutua se consigue garantizando que cuando un sistema está ejecutando una región crítica, en este momento ningún otro código puede entrar en la región crítica de la misma variable.



La parte sombreada son los momentos en los que un proceso entra en una región crítica. El proceso B intenta entrar en la región crítica (T₂), pero se bloquea porque está el proceso A dentro de su región crítica (T₁). Una vez el proceso A haya salido de la región crítica, el B podrá entrar en ella (T₃).

Soluciones para las carreras críticas: Para que una solución sea buena, tiene que cumplir 4 condiciones:

1. No puede haber dos procesos simultáneos en la región crítica (ya garantizado teniendo exclusión mutua).
2. No se pueden hacer suposiciones sobre velocidades o número de CPUs, como por ejemplo utilizando sleeps entre los hilos (ya que se pueden reducir mucho las posibilidades de carreras críticas, pero sigue habiendo posibilidades de que ocurra).
3. Un proceso no puede bloquear a otro sin estar en la región crítica. De esta forma, se debe garantizar que un proceso pueda entrar en la región crítica de una variable si no hay más procesos en ella.
4. Ningún proceso debe esperar indefinidamente al acceso a la región crítica, sino que debe tener la garantía que va a entrar en algún momento.

Las regiones críticas están asociadas a una variable, como por ejemplo la región crítica de la variable suma. Solo se debe garantizar la exclusión mutua para regiones de la misma variable, de forma que si el proceso A entra en la región crítica de la variable "resta", el proceso B puede entrar en la región crítica de la variable "suma".

3. Posibles Soluciones para las carreras críticas

EXCLUSIÓN MUTUA CON ESPERA OCUPADA

- Deshabilitando interrupciones: Garantizando que no puede haber interrupciones desde el momento en que un proceso entra en una región crítica, va a haber exclusión mutua. Es una solución muy delicada, ya que el sistema deja de atender todas las interrupciones, por lo que pueden surgir problemas debido a que hay procesos que se deben atender con prioridad máxima para evitar errores en el sistema. Los usuarios no suelen tener la posibilidad de hacer esto, sino que lo hace el kernel de sistema operativo en regiones críticas que ya se conocen previamente, de manera muy controlada. Además, para poder hacer esto la región crítica debe estar lo más acotada posible, para que de esta forma el tiempo en el que se deshabilitan las interrupciones sea mínimo. Por último, hay que considerar que Es una acción inútil en sistemas multiprocesadores o en sistemas multicore.

VARIABLES DE CANDADO (O BLOQUEO)

- Definimos una variable llamada “candado”, que está a 0 por defecto, y que va a valer esto mientras no haya una variable en la región crítica. Cuando se acceda a la región el candado se pone a 1 durante el tiempo en el que el proceso está en la región. Cuando salga de la misma, se vuelve a poner a 0.

No funciona, ya que lo único que se hizo fue mover el problema de la variable suma, a la variable candado, la cual ahora presenta una región crítica. No garantiza la exclusión mutua.

ALTERNANCIA ESTRICTA

```
While (TRUE) {  
    while (turno !=0) ;  
    region_critica();  
    turno=1;  
    region_no_critica();  
}
```

```
While (TRUE) {  
    while (turno !=1) ;  
    region_critica();  
    turno=0;  
    region_no_critica();  
}
```

Esta solución consiste en crear un código con dos lazos infinitos que comparten una variable llamada “turno”, que es compartida. En el primero de ellos, si “turno” no vale 0, el proceso se mantiene en espera activa. Cuando “turno” vale 0, sale del del lazo y se ejecuta el código dedicado a la región crítica. Una vez finaliza las instrucciones que pertenecen a la región crítica, el proceso modifica el valor de “turno” y lo pone a 1, para posteriormente seguir con ejecución de código que no pertenece a la región crítica.

Cuando “turno” valga 1 el segundo proceso saldrá de su lazo donde realizaba espera activa, y entrará en la región crítica de la variable compartida, y cuando finalice, pondrá la variable “turno” a 0, volviéndose a repetir el proceso cíclicamente.

Por otro lado, no se generará una región crítica en la variable turno, ya que cuando uno de los procesos modifica su contenido, el otro proceso se encuentra esperando en el lazo, por lo que no puede producir carreras críticas.

Hay que tener en cuenta que la espera activa es muy negativa, ya que es un derroche de recursos. Esto se debe a que se trata de un bucle esperando a que se modifique el contenido de una variable, por lo que se está consumiendo memoria para estar constantemente realizando ese chequeo continuamente. Además, si se trata de un sistema de un solo núcleo, se está utilizando todo el quantum en la espera.

Además, no cumple la condición 3, ya que se puede dar el caso de que uno de los procesos se encuentre esperando a que el otro modifique el valor de la variable “turno”, sin embargo, para que esto suceda, el otro proceso debe haber entrado en la región crítica, lo cual no tiene por qué suceder, de manera que el primero no puede entrar y se queda a esperando por el otro.



Escola
Técnica
Superior
de Enxeñaría



Grado en Ingeniería Informática

2º curso – 2º cuatrimestre

Sistemas Operativos 2

11-03-2021



Clase 5

Temas trabajados:

Herramientas para lidiar con carreras críticas

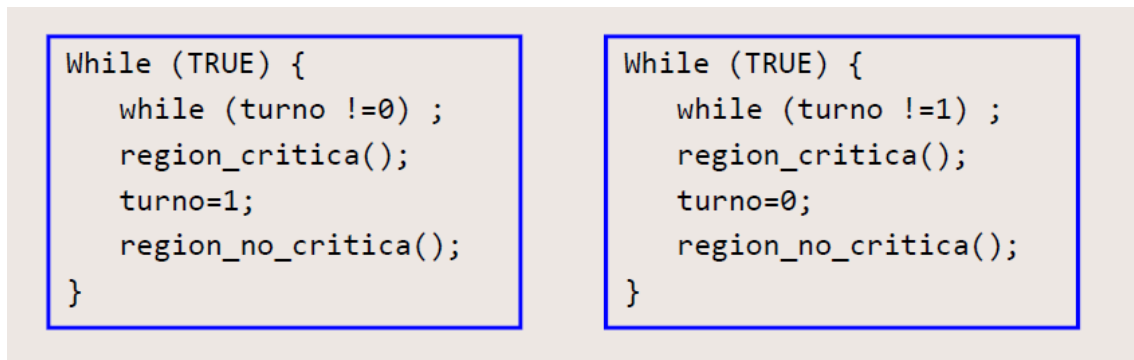
Índice

1. Evitación de Carreras Críticas.....	3
Introducción	3
Solución de Peterson.....	3
Instrucción TSL	5
Kahoot	5

1. Evitación de Carreras Críticas

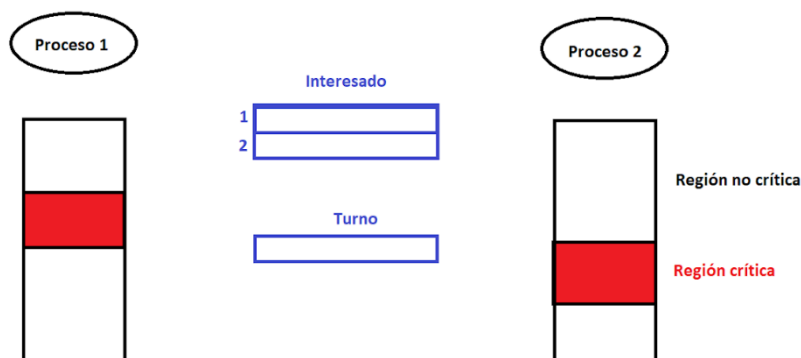
Introducción

Se comienza la clase repasando el concepto de alternancia estricta. En este método se ejecuta un bucle while infinito en el que una variable establece el turno para entrar en la región crítica. Esto es lo que se denomina una espera ocupada. Cuando un proceso sale del bucle while se ejecuta la región crítica y se pasa el turno al siguiente proceso. Esta manera de intentar solucionar carreras críticas tiene dos principales inconvenientes. La primera es que no se cumple la condición 3 que establece Tanenbaum para tener una solución buena frente a las condiciones de carrera (ningún proceso que se ejecute fuera de su región crítica puede bloquear otros procesos). Además, los procesos se ejecutarán al ritmo del más lento, pues tienen que estar esperando por él.



Solución de Peterson

Esta es una solución válida que cumple todos los requisitos establecidos por Tanenbaum. Es una solución puramente válida que no requiere nada especial (ni por parte del SO ni por parte del hardware). Vamos a ver un diagrama donde se muestra cómo se estructuran los procesos en esta solución.



Como podemos ver hay dos procesos, los cuales tienen regiones críticas en alguna parte de su código. Además, estos procesos tienen dos variables compartidas, una de ellas es la variable turno, que establece a quién le corresponde el turno de ejecución. Otra de las variables compartidas es el array Interesado, que tiene tantas posiciones de memoria como procesos estén usando la solución de Peterson en ese momento, en la posición 1 el proceso 1 manifestará su interés en tener la ejecución, lo mismo el proceso 2 en la posición 2 y así sucesivamente. La solución propiamente dicha se basa en que un proceso cuando va a entrar en la región crítica ejecuta una rutina o función

que se llama `enter_region()`. De manera análoga cuando sale de la región crítica ejecuta la función `leave_region()`. Veamos en detalle el código de dichas funciones.

```
#define FALSE 0
#define TRUE  1
#define N      2                /* number of processes */

int turn;                        /* whose turn is it? */
int interested[N];              /* all values initially 0 (FALSE) */

void enter_region(int process);  /* process is 0 or 1 */
{
    int other;                  /* number of the other process */

    other = 1 - process;        /* the opposite of process */
    interested[process] = TRUE; /* show that you are interested */
    turn = process;             /* set flag */
    while (turn == process && interested[other] == TRUE) /* null statement */ ;
}

void leave_region(int process)   /* process: who is leaving */
{
    interested[process] = FALSE; /* indicate departure from critical region */
}
```

La función `enter_region()` o `enter_region()` recibe como parámetro el número de proceso que lo está ejecutando (puede tener valor 0 o 1). La variable `other` representa el otro proceso, si el proceso que ejecuta este código es el proceso 0 `other` valdrá 1 y viceversa. Se pone la posición correspondiente al proceso del vector `Interesado` a `TRUE`, mostrando que el proceso tiene interés en ejecutar su RC. Se establece el turno al proceso y se ejecuta un `while` en el que la condición es que el turno esté asignado al proceso ejecutante y que el interés del otro proceso sea `TRUE`, es decir, se esperará mientras el `interested[other]` no sea `FALSE`.

Por otra parte lo único que hace la función `leave_region()` o `leave_region()` es poner `interested[process]` a valor `FALSE`, para que otros proceso que estén ejecutando la espera activa puedan entrar a la RC.

En esta solución también podrán ocurrir carreras críticas ya que se siguen usando variables compartidas, pero la manera en la que se estructura el código permite afirmar que estas carreras críticas son insignificantes si ocurren.

Esta solución, aunque implique espera activa, es válida y funciona para varios cores, algo que no sucedía si simplemente eliminábamos las interrupciones.

Instrucción TSL

Esta solución consiste en una instrucción máquina. El problema de las condiciones de carrera es tan importante que todos los lenguajes máquina tienen una instrucción de este tipo. Esta instrucción, del inglés “Test and Set Lock”, solo tiene un cometido y es resolver carreras críticas. Su funcionamiento se compone de una lectura y escritura de manera atómica (no puede haber interrupciones en el medio de la instrucción), por lo que necesita soporte hardware. Por ejemplo:

test R1, candado

Esta instrucción lo que hace es leer el valor de candado, escribirlo en el registro R1 y en candado escribir un 1.

Kahoot

Se hizo un kahoot, en el cual hubo 10 preguntas, estando la numero 5 mal corregida, la respuesta a esta pregunta es SI, y esta bien respondida en las fotos.

Este link a un OneDrive os dejará verlas.

[Primer Kahoot SisOp](#)



Escola
Técnica
Superior
de Enxeñaría



Grado en Ingeniería Informática

2º curso – 2º cuatrimestre

Sistemas Operativos II

18-03-2021



Clase 6

Temas trabajados:

Kahoot sobre semáforos y mutexes

Índice

1. Semáforos y mutexes	3
Ronda de preguntas	3
Kahoot	3

1. Semáforos y mutexes

Ronda de preguntas

Al principio de la clase Rivera abre una ronda de preguntas sobre los vídeos colgados en el Campus Virtual. Solo hay una pregunta: ¿Cuál es el protocolo para borrar semáforos o mutexes del kernel si se interrumpe la ejecución de un proceso que los utiliza?

Respuesta: Como las variables del kernel no se borran, permanecerán en el kernel a no ser que se ejecute una instrucción de borrado sobre ellos, por lo que puede ser una buena práctica intentar borrar los semáforos antes de crearlos para asegurarnos de que no existen. Otra alternativa es hacer un shellscript que borre los semáforos/mutexes al principio de un cierto código.

Kahoot

Posteriormente se procedió a realizar un Kahoot de 6 preguntas que no dio tiempo a acabar. Algunas de las preguntas están en el siguiente enlace [Kahoot3](#)

Faltan 3 preguntas ya que como no se acabó el Kahoot las preguntas no se almacenaron en la biblioteca y las únicas 3 preguntas que se pueden salvar son las que aparecen en la grabación de la clase.



Escola
Técnica
Superior
de Enxeñaría



Grado en Ingeniería Informática

2º curso – 2º cuatrimestre

Sistemas Operativos II

25-03-2021

Clase 07

Temas trabajados:

Variables de condición y adaptación del problema
productor-consumidor

Índice

1. INTRODUCCION A LAS VARIABLES DE CONDICIÓN	3
2. EJEMPLO DE USO DE VARIABLES DE CONDICIÓN	4
3. PROBLEMA DEL PRODUCTOR-CONSUMIDOR CON MUTEXES Y VARIABLES DE CONDICIÓN.....	5

1. INTRODUCCION A LAS VARIABLES DE CONDICIÓN

Los mutexes son usados para garantizar exclusión mutua, a través de dos funciones fundamentales, “mutex_lock” y “mutex_unlock”, funciones propias de Linux.

La función **lock** intenta entrar en una región crítica, de forma que si se encuentra accesible se sale de la función y se ejecuta la región crítica.

La función **unlock** se encarga de liberar una región crítica para que pueda ser usada posteriormente por otro proceso o hilo. Esta función nos garantiza exclusión mutua pues es una función atómica que no va a generar carreras críticas.

El lock y unlock están conectados por una variable compartida que es el mutex, que toma valores binarios.

Con estas funciones es complicado tener en cuenta ciertos aspectos como el tamaño del buffer en el problema del productor y el consumidor. Para ello se les añade una nueva funcionalidad a la que se le llama **variables de condición**. Tienen un comportamiento similar al de los semáforos y mutexes ya que es necesario crearlas y hay que borrarlas cuando terminan de ser usadas. Las funciones fundamentales de las variables de condición son “pthread_cond_wait” y “pthread_cond_signal”.

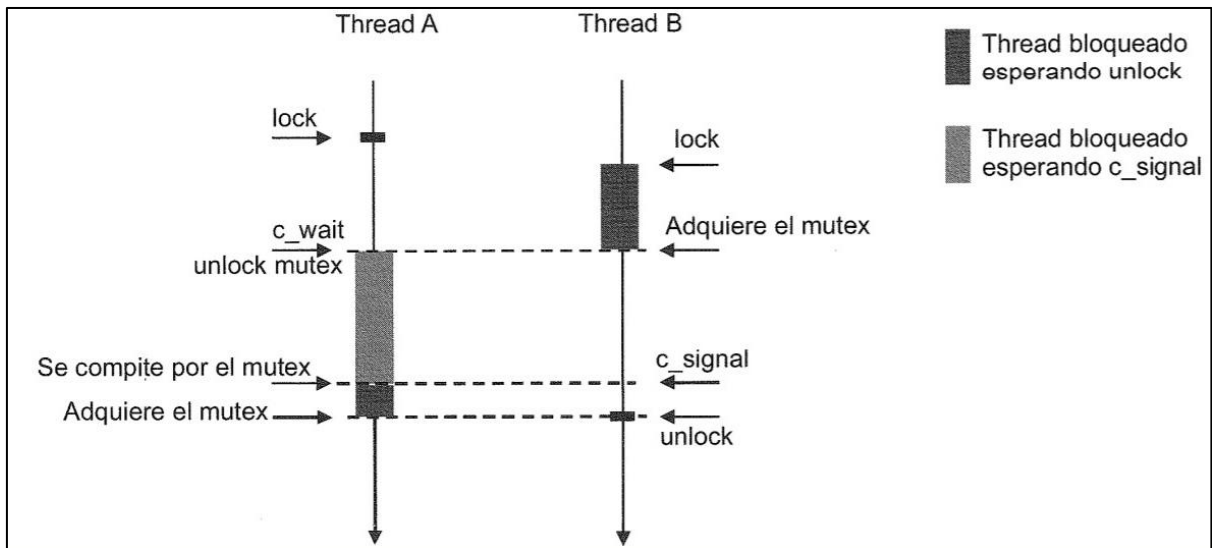
Wait es una función que hace que el proceso que la ejecute se auto bloquee y desbloquee al mutex asociado. Tiene que ser un proceso de la región crítica. Tiene como argumento el índice del mutex que desbloqueará. Cuando se ejecute esta función habrá dos procesos dentro de la región crítica, el proceso que ha entrado en la región crítica al desbloquearse su mutex, y el proceso que ejecuta el wait que queda bloqueado. El bloqueo terminará cuando la condición que ha hecho ejecutarse el pthread_cond_wait se satisfaga. En el momento en el que esto ocurra, el proceso saldrá del bloqueo a través de la función pthread_cond_signal que tendrá que ejecutar el otro proceso que aún se encuentra dentro de la región crítica.

Signal es una función que tiene el objetivo de desactivar la condición de la variable de condición para que un proceso que se encuentra bloqueado debido a ella, pueda desbloquearse. En el momento en el que se ejecuta habrá dos procesos desbloqueados dentro de la función crítica, y es tarea del sistema operativo decidir cuál de los dos procesos sigue.

Si se ejecuta un cond_signal sin que haya un proceso bloqueado por un cond_wait, ese signal se pierde al no haber un destinatario.

Otra función importante es “**pthread_cond_broadcast**” que se encarga de liberar a todos los procesos que se encuentren bloqueados por haber ejecutado un wait. La función signal sólo libera a un proceso, mientras que el broadcast libera a todos.

2. EJEMPLO DE USO DE VARIABLES DE CONDICIÓN



En esta figura se puede apreciar cómo se utilizan las variables de condición en la prevención de carreras críticas. Hay que tener en cuenta, que las variables de condición están asociadas a los mutexes, de tal manera que el uso de una variable de condición está pensado para que se realice en un mutex, dentro de la ejecución de una región crítica tras haber adquirido el acceso a ella después de un `mutex_lock`.

La figura representa dos hilos, A y B, que compiten por el acceso a una región crítica, y donde se puede apreciar que el hilo A, obtiene el acceso a esa región crítica antes que le hilo B, estando representado el tiempo en el eje y. En el momento en el que B intenta acceder a la región crítica, se queda bloqueado, tal y como se ve en la figura, representando el bloqueo de B con el color gris más oscuro.

Mientras B está bloqueado, el hilo A se encuentra en la región crítica realizando sus operaciones. Sin embargo, en un determinado momento puede darse el caso de que se necesite algo que no se ha realizado todavía para poder continuar su ejecución. En ese momento, ejecuta un `wait` de una variable de condición, mediante `pthread_cond_wait`, lo cual hace que se bloquee el proceso A y que se libere el mutex momentáneamente. En ese instante, como el mutex queda bloqueado, el hilo B, que estaba esperando para entrar en la región crítica, puede entrar en ella y realizar su ejecución.

En este momento se tiene que hay dos hilos en la región crítica, sin embargo, el hilo A se encuentra bloqueado. Hay que tener en cuenta que hay dos bloqueos distintos en esta representación. Por un lado, se tiene el bloqueo normal, realizado por un `lock`, y es el que sufría el proceso B al no poder entrar inicialmente en la región crítica ya que ya se encontraba el proceso A en ella. Por otro lado, el otro tipo de bloqueo, representado con un gris más claro, representa el bloqueo que se produce mediante una variable de condición la cual ha realizado un `wait`.

Supuestamente, en la ejecución dentro de la región crítica del proceso B, se resolverá el problema que provocó que el proceso A no pudiera continuar su ejecución, por lo que el

3. PROBLEMA DEL PRODUCTOR-CONSUMIDOR CON MUTEXES Y VARIABLES DE CONDICIÓN

proceso B es el encargado de ejecutar un `pthread_cond_signal`, lo cual despierta al proceso A. En este momento, se tienen a los dos hilos A y B dentro de la región crítica y desbloqueados, y es el sistema operativo el que escoge uno de los dos procesos para continuar su ejecución y bloquear al otro. En el caso de la figura, se ve que el proceso escogido es el B por lo que se puede apreciar que el proceso A se bloquea, mediante el bloqueo estándar de procesos, del color más oscuro, y B continua su ejecución. Cuando este termina, realiza un `unlock` del mutex, lo que libera a todos los posibles procesos que se encuentren esperando para acceder a la región crítica, y en particular desbloquea a A.

Un aspecto importante es que en el momento en el que se hace un `pthread_cond_signal` y se deja en manos del sistema operativo la elección de qué proceso va a continuar, puede provocar incertidumbre y situaciones imprevistas, por lo que se recomienda que en el proceso que se ejecuta, que en este caso es el B, se realice esta llamada cuando este proceso no tenga que realizar muchas operaciones posteriores, prácticamente antes de realizar la llamada a `unlock`.

Por último, hay que tener en cuenta que las funciones `pthread_cond_signal` se pierden si no hay procesos bloqueados por variables de condición, lo que puede provocar problemas si se da el caso, por ejemplo, que el proceso B ejecute el `signal` sin que A haya realizado el `wait`, por lo que A tendría que esperar a otro proceso para que lo desbloquee.

3. PROBLEMA DEL PRODUCTOR-CONSUMIDOR CON MUTEXES Y VARIABLES DE CONDICIÓN

A continuación, se presenta el problema del productor-consumidor planteado en ocasiones anteriores, pero resolviéndolo mediante el uso de mutexes y de variables de condición para evitar posibles carreras críticas.

El planteamiento de su resolución es prácticamente el mismo, sin embargo, para poder implementar mutexes y variables de condición se han tenido que realizar algunas modificaciones al código para su adaptación. Además, también se han introducido algunas modificaciones las cuales no son de especial relevancia en cuanto al problema y simplemente el autor las ha cambiado para su explicación.

En primer lugar, el buffer tiene tamaño 1, es decir, es simplemente una posición, por lo que el ritmo del productor y del consumidor se verá afectado e irán ambos al mismo ritmo.

Por otro lado, no se incluyeron las funciones de producir elemento y de consumir elemento en el productor y en el consumidor respectivamente, sino que se ha ignorado su implementación para esta visión teórica del problema.

3. PROBLEMA DEL PRODUCTOR-CONSUMIDOR CON MUTEXES Y VARIABLES DE CONDICIÓN

Otro cambio para tener en cuenta es que el consumidor, para indicar que ha consumido el único elemento presente en el buffer, escribe un 0 en el mismo. Este cambio no es del todo útil, ya que el productor no podrá enviar elementos cuyo valor sea 0 al buffer.

Por último, se puede apreciar la utilización de las funciones sobre variables de condición como `pthread_cond_wait` y `pthread_cond_signal` en lugar de las funciones `sleep` y `wakeup` descritas por Tanenbaum, lo cual permite el uso de variables de condición en el problema. Además, también se utilizan las funciones `pthread_mutex_lock` y `pthread_mutex_unlock` para bloquear o desbloquear el mutex que maneja el acceso a la región crítica tanto para el productor como para el consumidor.

```
#include <stdio.h>
#include <pthread.h>
#define MAX 1000000000 /* how many numbers to produce */
pthread_mutex_t the_mutex;
pthread_cond_t condc, condp;
int buffer = 0; /* buffer used between producer and consumer */

void *producer(void *ptr) /* produce data */
{
    int i;
    for (i = 1; i <= MAX; i++) {
        pthread_mutex_lock(&the_mutex); /* get exclusive access to buffer */
        while (buffer != 0) pthread_cond_wait(&condp, &the_mutex);
        buffer = i; /* put item in buffer */
        pthread_cond_signal(&condc); /* wake up consumer */
        pthread_mutex_unlock(&the_mutex); /* release access to buffer */
    }
    pthread_exit(0);
}
```

Código productor

```
void *consumer(void *ptr) /* consume data */
{
    int i;
    for (i = 1; i <= MAX; i++) {
        pthread_mutex_lock(&the_mutex); /* get exclusive access to buffer */
        while (buffer == 0) pthread_cond_wait(&condc, &the_mutex);
        buffer = 0; /* take item out of buffer */
        pthread_cond_signal(&condp); /* wake up producer */
        pthread_mutex_unlock(&the_mutex); /* release access to buffer */
    }
    pthread_exit(0);
}
```

Código consumidor



Escola
Técnica
Superior
de Enxeñaría



Grado en Ingeniería Informática

2º curso – 2º cuatrimestre

Sistemas Operativos II

08-04-2021



Clase 08

Temas trabajados: Kahoot sobre semáforos, mutexes y variables de condición. Repaso vídeos de pases de mensajes y monitores

Índice

1. Kahoot sobre semáforos, mutexes y variables de condición.....	3
Soluciones y comentarios.....	4
2. Monitores	4
3. Pase de mensajes	4
El problema del productor-consumidor resuelto con paso de mensajes.....	5

1. Kahoot sobre semáforos, mutexes y variables de condición

1. Si el tamaño del buffer es N. ¿Qué valores puede tomar el semáforo “vacías” en la solución vista en clase?

- A) Entre 0 y N-1 inclusive
- B) Entre 0 y N inclusive
- C) Entre -N y N inclusive
- D) Solamente 0 o 1

2. ¿Por qué en la definición de la función up, el proceso bloqueado por el correspondiente down debe volver a ejecutarse?

- A) Para volver a ganar el acceso a la región crítica
- B) No haría falta si el S.O lo desbloquea solo a él, y a ningún otro proceso
- C) Quizás el que ejecuta el up entra enseguida en la región crítica otra vez
- D) Todas las anteriores son ciertas

3. ¿Qué pasa si cuatro procesos quieren ejecutar un down al mismo tiempo en cores diferentes?

- A) Quizás se bloquean los cuatro
- B) Quizás no se bloquee ninguno
- C) A y B son correctas
- D) Seguro que no se bloquea ninguno

4. En el código ensamblador con TSL para la función unlock, ¿dónde se libera al hilo que está bloqueado?

- A) Eso falta en el código
- B) El bloqueado realmente está en espera activa, y ahora puede salir de ella
- C) La función unlock no desbloquea a nadie, así que no hace falta
- D) Eso lo hará el SO, y no hay necesidad de programarlo

5. Respecto a la función trylock...

- A) Casi siempre es mejor que lock porque no bloquea al proceso
- B) Es como un lock pero sin bloquear al proceso
- C) Sirve si hay algo que hacer alternativo a entrar en la región crítica
- D) Sirve para intentar bloquear a otro proceso indicado cómo parámetro

Soluciones y comentarios

Soluciones: 1b, 2d, 3c, 4b, 5c.

Comentarios realizados por Rivera durante la corrección de las preguntas:

Pregunta 2:

Down para volver a ganar la región crítica: Si hay varios procesos bloqueados por ese down, una vez resuelto el tema solamente uno de ellos debe seguir. No pueden seguir más de uno porque estamos en la región crítica. Por ende, la primera es correcta.

Esto se resuelve si el SO, cuando se ejecuta el up, solamente desbloquea a 1 de los procesos en espera y a los demás no los libera de ese down. Solamente un proceso sigue.

En caso de la opción c, el que gane el down es el que se ejecuta.

La opción d también es correcta, puede ocurrir que el proceso que ejecuta el up siga avanzando y vuelva a entrar en la región crítica. Por tanto, todas son correctas

Pregunta 3:

Quizás no se bloquean los semáforos si el valor es muy alto y puede que se bloqueen los 4 o ninguno. La última opción no sería correcta debido a que el down está pensado para que se bloquee por lo menos un proceso.

Pregunta 4:

Si se desea que el SO realice una acción en un determinado momento se debe realizar un syscall, no hay otra forma, a no ser que exista una interrupción, que no es el caso en esta pregunta.

El código del lock, implica una espera activa. Estamos en el lazo hasta que el valor del mutex es 0, en ese momento se sale de la espera activa. El unlock, pone un 0 en mutex.

Pregunta 5:

El tryLock bloquea o no al proceso, depende de lo que valga el valor del semáforo. El hecho de que se diga que casi siempre es mejor no es correcto, de echo tienen finalidades muy diferentes. El tryLock casi nunca se usa por lo que se debe descartar la respuesta c.

Con respecto a la b, un tryLock no es como un lock, un lock bloquea el proceso.

La d no tiene ningún sentido, un tryLock no bloquea otro proceso ni tiene un identificador como parámetro.

2. Monitores

Es un mecanismo de alto nivel: menor posibilidad de error, pero menos versátil. El monitor tiene una estructura similar a la de un objeto en el que se definen procedimientos (las regiones críticas) con exclusión mutua garantizada (lo hace el compilador seguramente con semáforos o mutexes). Para hacerlos más versátiles suelen ir acompañados del uso de variables de condición.

3. Paso de mensajes

Es la única opción para sistemas distribuidos (memoria no compartida). Estos sistemas desacoplados pueden incluso tener sistemas operativos diferentes, por lo que el paso de mensajes es el único mecanismo disponible en estas circunstancias. Esto se consigue a través de funciones de la librería mpi (message passing interface), con dos funciones clave: send y receive. Los sistemas cuentan con buffers de envío y recepción de mensajes, por lo que incluso se pueden realizar distintos tipos de envíos. Por ejemplo, el envío bloqueante bloquea al proceso que ejecuta

el send hasta que el mensaje llega a su destino. También existe el envío síncrono, que bloquea al proceso que realiza el send hasta que llegue el mensaje y sea recibido por el proceso receptor, es decir, salga del buffer de recepción y se ejecute el receive. Existen mecanismos para garantizar la recepción de los mensajes, como puede ser el uso de acknowledgements y autenticación.

El problema del productor-consumidor resuelto con paso de mensajes

```
#define N 100                                /* number of slots in the buffer */

void producer(void)
{
    int item;
    message m;                               /* message buffer */

    while (TRUE) {
        item = produce_item();               /* generate something to put in buffer */
        receive(consumer, &m);               /* wait for an empty to arrive */
        build_message(&m, item);             /* construct a message to send */
        send(consumer, &m);                  /* send item to consumer */
    }
}

void consumer(void)
{
    int item, i;
    message m;

    for (i = 0; i < N; i++) send(producer, &m); /* send N empties */
    while (TRUE) {
        receive(producer, &m);               /* get message containing item */
        item = extract_item(&m);             /* extract item from message */
        send(producer, &m);                  /* send back empty reply */
        consume_item(item);                  /* do something with the item */
    }
}
```

Como podemos ver en el código de Tanenbaum, el consumidor le manda al productor N mensajes vacíos, para indicarle que el buffer es de tamaño N. El consumidor cada vez que recibe un mensaje vacío responde mandando un producto, y cuando el consumidor recibe el producto lo indica enviando un nuevo mensaje vacío.



Grado en Ingeniería Informática

2º curso – 2º cuatrimestre

Sistemas Operativos II

15-04-2021

Clase 09

Temas trabajados:

Kahoot y Problema de los filósofos

Índice

1- KAHOOT	3
Pregunta 1.	3
Pregunta 2.	3
Pregunta 3.	4
Pregunta 4.	4
Pregunta 5.	5
2- PROBLEMAS CLÁSICOS DE COMUNICACIÓN ENTRE PROCESOS.....	5
2.1 - PROBLEMA DE LOS FILÓSOFOS	5

1- KAHOOT

Pregunta 1.

Una sincronización muy compleja, con muchos condicionantes, entre varios procesos la programaría con ...

Mostrar contenido

Finalizar juego

▲ Semáforos	✓
◆ La solución de Peterson	✗
● Monitores	✗
■ Señales	✗

Explicación:

Si se trata de un código complejo, con muchos condicionantes y situaciones que requieren un control, los semáforos ofrecen una versatilidad elevada, permitiendo manejar aquellas variables que se usan como condicionantes, bloqueando a los procesos cuando se requiere para evitar carreras críticas. Los monitores no sería una respuesta correcta ya que se trata de un mecanismo de alto nivel que no ofrece mucha versatilidad, por lo que en programas complejos no serían útiles. La solución de Peterson presenta problemas ocasionalmente, por lo que no valdrían como solución correcta. Por último, las señales no sirven para la sincronización entre procesos.

Pregunta 2.

Si solo quiero acceso exclusivo a una región crítica, sin más condicionantes, lo más sencillo para programarlo es ...

Mostrar contenido

Finalizar juego

▲ Semáforos	✗
◆ La solución de Peterson	✗
● Monitores	✓
■ Señales	✗

Explicación:

De manera análoga al apartado anterior, los monitores sirven para la sincronización entre procesos garantizando la exclusión mutua en las regiones críticas, sin embargo, son poco versátiles, por lo que se suelen usar en problemas sencillos, como es el caso.

Pregunta 3.

En la solución al problema del productor-consumidor con pase de mensajes vista en clase, ¿quién envía mensajes?

Mostrar contenido

Finalizar juego

Si siguiente

▲ El productor	✗
◆ El consumidor	✗
● Ambos	✓
■ El planificador	✗

Explicación:

Inicialmente, en el problema del productor consumidor con pase de mensajes, el productor envía mensajes al consumidor con la información producida para que el consumidor la reciba y la consuma. Sin embargo, en la solución propuesta por Tanenbaum, el consumidor envía una serie de mensajes vacíos al productor con el objetivo de llevar un control preciso del tamaño de los buffers, lo cual permite la ejecución desacoplada del productor y del consumidor, es decir, sin tener que esperar uno por el otro para producir y consumir cada ítem.

Pregunta 4.

Durante la ejecución de la solución, ¿quién envía más mensajes?

Mostrar contenido

Finalizar juego

Si siguiente

▲ El productor	✗
◆ El consumidor	✓
● Los dos por igual	✗
■ Es imposible saberlo con seguridad	✗

Explicación:

El consumidor es el primero que envía tantos mensajes como tamaño tenga el tamaño del buffer, que sirve para llevar el desacoplamiento entre el productor y el consumidor. A partir de ese momento, el productor comenzará a enviar mensajes cada vez que produzca y envíe un ítem, y el consumidor cada vez que consuma, por lo que los mensajes enviados por el productor no superarán nunca a los enviados por el consumidor, a pesar de que se pueden llegar a igualar en un

determinado momento si se llena el buffer, pero los cuales serán contestados cuando el consumidor los consuma.

Pregunta 5.



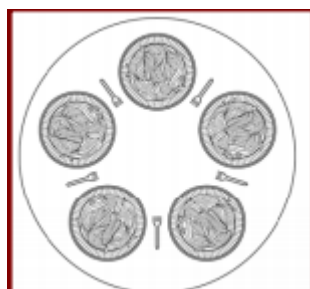
Explicación:

En las barreras de sincronización, a medida que los procesos involucrados en el programa van llegando a la barrera, se van bloqueando. Cuando llega el último proceso, en vez de bloquearse también, lo que hacen las barreras es desbloquear a todos los procesos bloqueados, por lo que continúan toda su ejecución, tanto el último como los demás. Consecuentemente, las barreras de sincronización bloquean a todos los procesos menos a uno, el último, ya que este no se va a bloquear y va a provocar el desbloqueo de los demás.

2- PROBLEMAS CLÁSICOS DE COMUNICACIÓN ENTRE PROCESOS

2.1 - PROBLEMA DE LOS FILÓSOFOS

Este problema se denomina a un conjunto de procesos como “**filósofos**” que realizan un trabajo diferente del resto de filósofos denominado “**pensar**”. Durante el tiempo en el que cada filósofo esté realizando su trabajo no existen carreras críticas, ya que son independientes entre sí. En la siguiente imagen se puede ver una representación gráfica de este problema:



Cada filósofo tiene delante un plato con dos tenedores, uno a cada lado. Estos tenedores serán el recurso compartido entre los diferentes filósofos. En el momento en el que un filósofo va a coger los tenedores, entra en la región crítica. Para que un filósofo pueda comer, es necesario que tenga los dos tenedores, por lo que los otros dos filósofos adyacentes no podrán comer al faltarles uno de sus tenedores. Además, puede darse el caso de que dos filósofos estén comiendo al mismo tiempo, ya que, si estos no están contiguos, pueden coger los tenedores de sus vecinos. Cuando un filósofo termina de comer devuelve los dos tenedores, y los otros filósofos podrán pasar a comer. Consecuentemente, las posibles carreras críticas se pueden producir cuando se van a coger los tenedores y cuando se van a dejar. Estas dos regiones críticas se corresponden a las funciones de `take_forks` y de `put_forks`.

```
#define N          5                /* number of philosophers */
#define LEFT      (i+N-1)%N        /* number of i's left neighbor */
#define RIGHT     (i+1)%N          /* number of i's right neighbor */
#define THINKING  0                /* philosopher is thinking */
#define HUNGRY    1                /* philosopher is trying to get forks */
#define EATING    2                /* philosopher is eating */
typedef int semaphore;             /* semaphores are a special kind of int */
int state[N];                     /* array to keep track of everyone's state */
semaphore mutex = 1;              /* mutual exclusion for critical regions */
semaphore s[N];                   /* one semaphore per philosopher */

void philosopher(int i)            /* i: philosopher number, from 0 to N-1 */
{
    while (TRUE) {                 /* repeat forever */
        think();                   /* philosopher is thinking */
        take_forks(i);             /* acquire two forks or block */
        eat();                     /* yum-yum, spaghetti */
        put_forks(i);              /* put both forks back on table */
    }
}
```

En esta solución al problema se usan dos semáforos compartidos. Por un lado, un mûtex para controlar el acceso a la región crítica, y otro un array (`s[N]`), el cual sirve para indicar la situación en la que se encuentra cada uno de los procesos filósofos. Además, se tiene un array compartido (`state[N]`), el cual tienen tantas entradas como número de filósofos haya, y que nos indica en qué estado está cada filósofo: **pensando**, **comiendo**, y **hambriento**. Cuando un filósofo está hambriento, representa al momento en el que un filósofo tiene la intención de comer, el cual puede ser corto si adquiere rápidamente los tenedores, o largo si no es capaz de adquirirlos. En la solución de Tanenbaum, cuando un proceso se encuentra hambriento, se encuentra bloqueado, a la espera de adquirir los tenedores y desbloquearse.

Las principales funciones para tratar el problema son las de coger y dejar los tenedores:

- **Take_forks**

```
void take_forks(int i) /* i: philosopher number, from 0 to N-1 */
{
    down(&mutex); /* enter critical region */
    state[i] = HUNGRY; /* record fact that philosopher i is hungry */
    test(i); /* try to acquire 2 forks */
    up(&mutex); /* exit critical region */
    down(&s[i]); /* block if forks were not acquired */
}
```

Para coger el tenedor, como se va a acceder a variables compartidas, se entra en la región crítica, por lo que se tiene que hacer un down del mútex para adquirir la exclusión mutua sobre ese proceso filósofo, o bloquearse si ya hay otro proceso en dicha región, teniendo que esperar a que acabe. A continuación, como el filósofo va a intentar coger los tenedores, se declara a ese filósofo como hambriento y se llama a test, la cual intentará coger el tenedor de la izquierda y de la derecha. Independientemente de lo conseguido en la función test, se desbloquea el mútex para que otros procesos puedan acceder a la región crítica, y posteriormente se realiza un down del semáforo del propio filósofo. Si en test se ha conseguido obtener los dos tenedores, dicho semáforo valdrá 1, por lo que al hacer down pasará a 0, no se bloqueará y seguirá su ejecución, comiendo posteriormente ejecutando eat en el lazo principal. Sin embargo, si no se han adquirido los dos tenedores, este semáforo valdrá 0, por lo que al hacer down el proceso de este filósofo se bloqueará, y tendrá que esperar a que otro proceso, u otro filósofo, realice un up y haga desbloquear a este mútex.

- **Test**

```
void test(i) /* i: philosopher number, from 0 to N-1 */
{
    if (state[i] == HUNGRY && state[LEFT] != EATING && state[RIGHT] != EATING) {
        state[i] = EATING;
        up(&s[i]);
    }
}
```

En la función test, se comprueba si el filósofo está hambriento y si el filósofo de la izquierda y el de la derecha no están comiendo, lo cual implicaría que los dos tenedores del filósofo están disponibles. En el caso de que se cumpla la condición, a través de la variable state de uso compartido, el estado del filósofo pasa a ser “comiendo”, para que los filósofos contiguos ya sepan que me encuentro comiendo. Finalmente se hace un up sobre el semáforo del filósofo, asociado al down de la función take_forks, para que el filósofo no se bloquee como se comentó previamente. En el caso de que alguno de los filósofos de la derecha o de la izquierda se encuentren comiendo, el proceso se bloqueará al no ejecutarse el up y no poder obtener los tenedores.

- **Put_forks**

```
void put_forks(i)                /* i: philosopher number, from 0 to N-1 */
{
    down(&mutex);                /* enter critical region */
    state[i] = THINKING;         /* philosopher has finished eating */
    test(LEFT);                  /* see if left neighbor can now eat */
    test(RIGHT);                 /* see if right neighbor can now eat */
    up(&mutex);                  /* exit critical region */
}
```

Para la función de dejar tenedores, también se van a usar a variables compartidas, por lo que hay se va a acceder a la región crítica. Consecuentemente, se hace un down del m utex para avisar al resto de que va a entrar en la regi n cr tica y asegurar la exclusi n mutua. A continuaci n, el estado pasa a ser “pensando”, ya que se acab  de comer y se van a dejar los tenedores. Posteriormente, se hace una llamada a la funci n test para el fil sofo de la izquierda y el de la derecha, en vez de sobre s  mismo. De esta manera lo que se va a conseguir es informar a los fil sofos contiguos de que se ha terminado de comer y que se va a dejar los tenedores. Por lo tanto, si uno de ellos se encuentra hambriento, indica que est  bloqueado esperando a los tenedores de sus vecinos. Se hace una comprobaci n del estado de los fil sofos contiguos de este, el cual uno de ellos ser  el que estaba dejando los tenedores e invoc  a la funci n test. Con lo cual, si ahora el fil sofo adyacente que estaba esperando por comer puede acceder tanto al tenedor que acaba de dejar el primero, como al del otro lado, se cambiar  el estado a comiendo y se har  un up y se desbloquear  a ese proceso que se encontraba en el down de la funci n de take_forks, haciendo que pueda comer. Sin embargo, si el fil sofo del otro lado se encuentra comiendo, este deber  seguir esperando a que dej  los tenedores para poder desbloquearse.



Escola
Técnica
Superior
de Enxeñaría



Grado en Ingeniería Informática

2º curso – 2º cuatrimestre

Sistemas Operativos II

22-04-2021



Clase 10

Temas trabajados:

Tema 2

Índice

1. Introducción	3
2. Recursos	3
3. Adquisición de recursos	4
4. Condiciones de los interbloques	5
5. Representación de interbloques mediante grafos.....	6
6. Soluciones de los interbloques	8

1. Introducción

Los interbloqueos son situaciones en las que **al menos dos procesos** están bloqueados indefinidamente al entrar en conflicto sus diferentes necesidades. Estas situaciones se producen cuando los procesos compiten por el uso de recursos compartidos, ya sean recursos hardware (acceso a impresora, acceso a dispositivo de E/S) o software (acceso a una determinada posición de memoria, entradas de base de datos...), comunicándose y sincronizándose entre sí.

Este tipo de contratiempos tienen una solución muy compleja en términos de tiempo de computación, lo que hace que en la práctica no se suelen remediar, de manera que, en bastantes casos se aplican medidas drásticas (por ejemplo, terminar la ejecución de los procesos) para lidiar con ellos.

2. Recursos

Los recursos son objetos o entidades que el Sistema Operativo otorga a los procesos. Estos pueden ser tanto hardware como software (como al trabajar con posiciones de memoria compartidas).

Existen dos tipos de recursos:

- **Apropiativos:** Un recurso es apropiativo si se le puede quitar a un proceso sin provocar consecuencias graves. Por ejemplo: el uso de memoria principal, el cual se otorga a un proceso diferente al producirse un cambio de contexto.
- **No apropiativos:** Un recurso es no apropiativo cuando no se le puede quitar a un proceso sin ocasionar efectos negativos. Por ejemplo: una impresora, ya que si se le otorga a un nuevo proceso, entonces se van a mezclar los contenidos a imprimir con los del proceso que antes la poseía. Otro ejemplo sería el acceso a una variable compartida cuando pueden ocurrir carreras críticas (si un proceso está en la región crítica no se le puede quitar el acceso a dicho recurso de manera drástica. El proceso debe haber acabado de operar en ella, lo cual se garantiza mediante el uso de mutexes o semáforos).

Son este tipo de recursos los que presentan el problema de los interbloqueos.

Cuando un proceso requiere un recurso lo va a solicitar ejecutando los siguientes pasos:

- Solicita el recurso al S.O. (lo que equivale a ejecutar *mutex_lock* o *down*)
- Cuando adquiera el recurso, lo usa de manera exclusiva y sin que se le pueda extraer hasta terminar su operación con el mismo.
- Al acabar su operación con el recurso, lo libera mediante una llamada al sistema (lo que equivale a ejecutar *mutex_unlock* o *up*)

Para que se produzca un interbloqueo se deben suceder varias solicitudes de recursos continuadas, como se observa en el código de la derecha de la imagen inferior. Códigos como el de la izquierda nunca conducirían a este tipo de situaciones:

```
typedef int semaphore;
semaphore resource_1;
```

```
void process_A(void) {
    down(&resource_1);
    use_resource_1( );
    up(&resource_1);
}
```

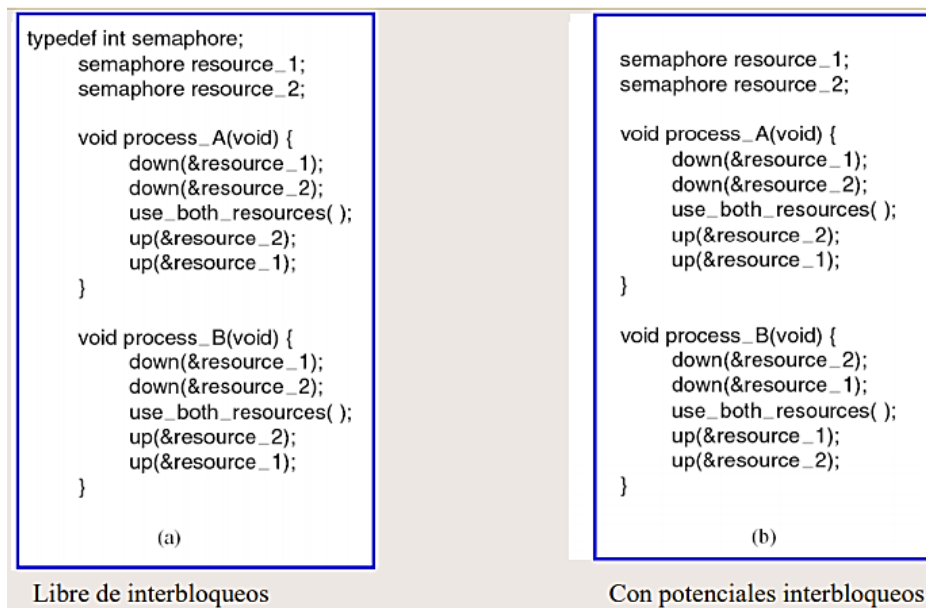
```
typedef int semaphore;
semaphore resource_1;
semaphore resource_2;
```

```
void process_A(void) {
    down(&resource_1);
    down(&resource_2);
    use_both_resources( );
    up(&resource_2);
    up(&resource_1);
}
```

De esta manera se debe completar la definición de interbloqueo proporcionada en el apartado anterior, de forma que para que exista un interbloqueo se necesitan **al menos dos procesos y como mínimo dos recursos** a los cuáles intentará acceder cada proceso de manera simultánea.

3. Adquisición de recursos

En la imagen inferior se muestra el interbloqueo más sencillo que se puede dar: dos procesos donde cada uno intenta adquirir dos recursos compartidos a la vez.



Ambos procesos, A y B, poseen la misma secuencia de funciones, sin embargo, se desconoce el orden de ejecución de los procesos, ya que varía dependiendo de las decisiones del planificador. Esto complica la detección de este tipo de problemas, que además se producen con muy baja frecuencia (solamente algunos órdenes de ejecución producen interbloqueos, puesto que, por ejemplo, no se produciría ningún interbloqueo en caso de que A se ejecutase enteramente antes que B).

En la primera figura de la imagen anterior no se producirá ningún problema del estilo. El primer *down* lo ejecutará o A o B (el primero en hacerlo), lo cual sólo se puede afirmar gracias a que esta

función es atómica. Por tanto, el proceso que realice el *down* primero podrá continuar con el código sin que interfiera el otro proceso, puesto que este se bloqueará.

El problema surge en la figura de la derecha, debido a que el orden de los *down*, en este caso particular, es diferente en el proceso A con respecto al B:

Si A ejecuta el *down* del recurso 1 con éxito, a continuación, el del recurso 2 y suponemos que pierde la CPU, de manera que B ejecuta el *down* del recurso 2, al estar libre dicho recurso, este segundo proceso lo obtendrá. Al intentar hacer el *down* del recurso 1, B se quedará bloqueado (ya que lo solicitó A primero) y cuando A vuelva a adquirir la CPU y quiera acceder al recurso 2, también se bloqueará (ya que lo solicitó B primero). De esta forma, ambos procesos se bloquean irremediablemente, puesto que ninguno de ellos podrá ejecutar el *up*. La solución en este sencillo caso consistiría en invertir el orden de los *down*, sin embargo, en casos más complejos resulta difícil de aplicar, ya que muchas veces los recursos están en el Sistema Operativo, y o todos tiene claro el orden en el que se adquieren o se pueden dar estas circunstancias (en caso de que dos programas accedan cada uno a varios recursos hay que llegar a un acuerdo, lo cual no suele ser viable porque implicaría llegar a un consenso sobre todos los posibles recursos).

Una vez se determina que el problema existe se puede solucionar fácilmente, el principal problema es detectarlos, ya que solo se dan en determinadas ocasiones y, además, los procesos se bloquean teniendo que abortarlos, por lo que si se implican muchos procesos, el S.O. puede bloquearse por completo (lo que requiere de la drástica solución de apagar el sistema y encender de nuevo, perdiendo todas las ejecuciones).

Por este motivo, otra definición que pueden tomar los interbloqueos sería la siguiente:

Un conjunto de procesos están en interbloqueo si cada uno está esperando algo que solo puede ser ocasionado por otro proceso del conjunto. (es decir, que varios procesos estén bloqueados no quiere decir que se trate de un interbloqueo).

4. Condiciones de los interbloqueos

Se necesitan cuatro condiciones para que ocurra un interbloqueo, si alguna no se cumple entonces se garantiza que el problema no se trata de un interbloqueo:

- **Condición de exclusión mutua:** Los recursos implicados se asignan de manera exclusiva.
- **Condición de contención y espera:** En el sistema tiene que existir la posibilidad de que un proceso que tenga un recurso pueda acceder a otro recurso, también de manera exclusiva, sin haber liberado el primer recurso retenido: en términos de mutexes consiste en realizar varios *mutex_lock* anidados (esta idea de poder realizar llamadas a *mutex_lock* anidadas implica que en funciones como *cond_wait*, *cond_broadcast* o *cond_signal* haya que indicar el recurso respecto al cual se realiza).
- **Condición no apropiativa:** Los recursos no se pueden extraer de cualquier forma, de manera que si un proceso entró en la región crítica, el S.O. no puede quitarle el acceso a dicha región sin que se trate de una situación grave.

5. Representación de interbloqueos mediante grafos

- **Condición de espera circular:** Varios procesos que adquieren varios recursos participan en un ciclo. Se ahondará en esta cuestión posteriormente.

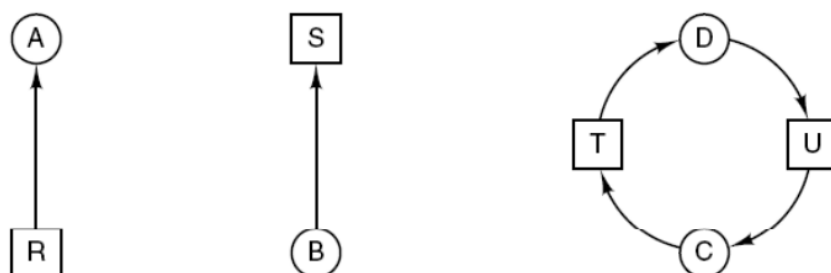
Hay Sistemas Operativos que están diseñados de tal forma que una de las cuatro condiciones no se cumpla nunca por política (por ejemplo: se establece que todos los recursos sean apropiativos, que no se permita la adquisición de más de un recurso a la vez...). Si eso se consigue, el S.O. está libre de interbloqueos, pero a cambio se obtienen efectos muy negativos: se hace más difícil la programación, el sistema es menos eficiente, etc. Por eso, sólo en los casos en los que el S.O. no debe entrar en interbloqueo bajo ningún concepto (como durante el pilotaje de un avión, en las expediciones a marte o en el sistema de control de una central nuclear) se tomarán ese tipo de medidas.

A modo de ejemplo de interbloqueo se realizará una analogía con una rotonda en la cual se produjo un atasco. Para ello se puede suponer que los procesos vienen representados por los coches y los recursos, por los carriles de la rotonda. Un coche A está en un carril, por lo que lo tiene “bloqueado”. Por un lado, ese coche A quiere otro carril que está ocupado por otro coche, B, que además de tener el carril que el coche A desea, a la vez quiere otro carril ocupado por un coche C, y así sucesivamente. Solo en el caso en el que esa cadena acabe en un ciclo se producirá un interbloqueo (lo que hace referencia a la cuarta condición introducida previamente).



5. Representación de interbloqueos mediante grafos

Para modelar la presencia de interbloqueos se emplearán grafos dirigidos, de manera que en un grafo se representarán dos tipos de nodos: los círculos representarán procesos y los cuadrados, recursos.



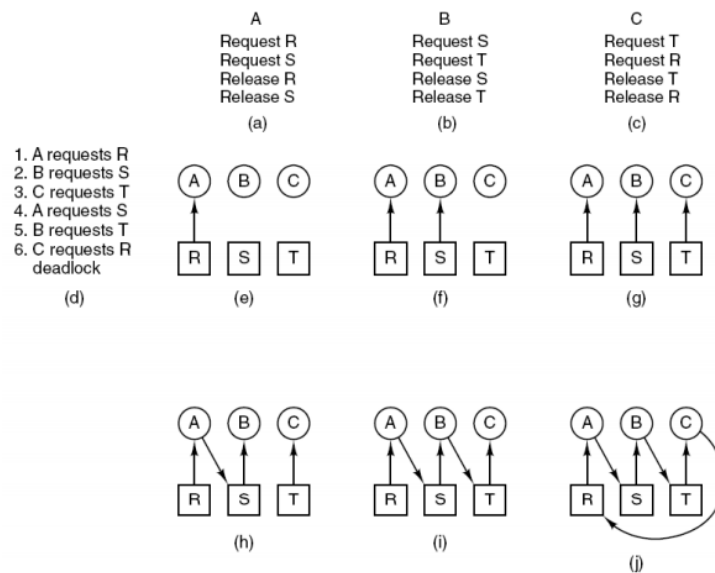
5. Representación de interbloqueos mediante grafos

- En la imagen de la izquierda se muestra una flecha dibujada desde el recurso al proceso, lo que expresa que proceso ya tiene dicho recurso.

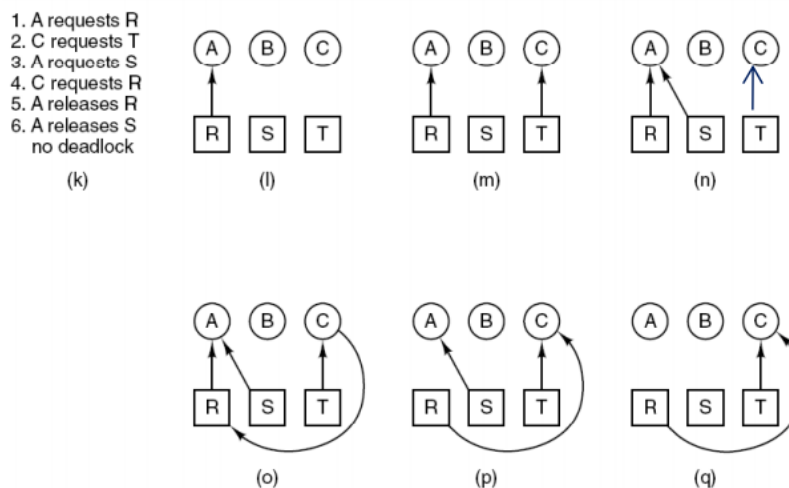
- La figura del medio muestra el caso en el que la flecha va desde el proceso al recurso, lo que expresa que el proceso quiere acceder al recurso de manera exclusiva, pero que aún no lo tiene (en términos de mutexes se entendería que el proceso ejecutó el *mutex_lock* de ese recurso y quedó bloqueado hasta que se le proporcione).

- En la figura de la derecha se observa el fenómeno de interbloqueo representado mediante un grafo: el proceso D tiene el recurso T y quiere el recurso U, mientras que el proceso C tiene el recurso U y quiere el recurso T. Es decir, un interbloqueo se corresponde con un ciclo en este tipo de grafos anteriormente presentados. Este es el ciclo más simple que se puede dar: dos procesos y dos recursos involucrados en un ciclo.

Un ejemplo más complejo sería el que muestra la imagen siguiente, en donde tres procesos solicitan simultáneamente tres recursos:



Como se puede observar, se termina formando un ciclo, lo cual indica que la secuencia de pasos representada a la izquierda produjo un interbloqueo. Por el contrario, la imagen inferior sería una secuencia diferente del mismo caso, en la cual no se produce ningún interbloqueo, al no observarse ningún ciclo.



6. Soluciones de los interbloqueos

Existen cuatro estrategias para lidiar con el problema:

- **Ignorar el problema:** debido a que las estrategias de resolución son demasiado costosas y a que, generalmente, la pérdida no es muy grave, esta es la solución habitual y la que adoptan sistemas como Windows o Linux. Para salir del interbloqueo, el usuario deberá matar al proceso o apagar el sistema.

Un algoritmo que implementa esta solución es el **algoritmo del avestruz**.

- **Detección y recuperación:** Algunos S.O. son capaces de detectar que hay un interbloqueo (por ejemplo, construyendo un grafo y detectando si se produce algún ciclo) y luego aplicar alguna técnica para resolverlo, las cuales son drásticas (matar a un proceso, entre otras).

- **Evitarlos dinámicamente:** Es la mejor solución para evitar interbloqueos. Antes de darle un recurso a algún proceso, se estudia si eso puede ocasionar problemas, y en caso afirmativo, no se le otorga dicho recurso al proceso solicitante, permaneciendo bloqueado. Resulta una solución muy complicada, ya que hay que analizar la situación de cada proceso con respecto a todos los recursos, de manera que, además, aunque el recurso esté libre, el sistema puede decidir no proporcionarlo porque podría dar problemas a posteriori.

En este caso, el **algoritmo del banquero** evita dinámicamente la probabilidad de ocurrencia de interbloqueos.

- **Prevención aplicando políticas de manejo de los recursos:** consiste en evitar que se suceda alguna de las cuatro condiciones necesarias para que se produzca un interbloqueo.



Escola
Técnica
Superior
de Enxeñaría



Grado en Ingeniería Informática

2º curso – 2º cuatrimestre

Sistemas Operativos II

03-05-2021



Clase 11

Temas trabajados:

Detección de Interbloqueos, Recuperación de
Interbloqueos

Índice

1.	Detección de interbloqueos	3
2.	Detección de interbloqueo con varios recursos de cada tipo	4
3.	Recuperación de un interbloqueo	6
4.	Evitar Interbloqueos	6

2. Detección de interbloqueo con varios recursos de cada tipo

Hasta ahora, habíamos considerado que un recurso era único y solo se podía asociar a un proceso. Sin embargo, esto es un caso particular ya que, generalmente, un recurso puede ser accedido por varios procesos al mismo tiempo hasta un número máximo definido. Este número se denomina instancias de un recurso. El siguiente algoritmo que se explicará está relacionado con este tipo de situaciones, en las que un recurso se puede encontrar asociado a varios procesos simultáneamente.

Para comprender este, y futuros algoritmo, es necesario entender y definir una serie de conceptos clave los conforman:

- Vector de recursos existentes (E): SE trata de un vector de m entradas donde cada entrada representa un recurso del sistema. El número de cada entrada consiste en el número de instancias de cada recurso. Se trata de un vector constante ya que es meramente informativo, y no variará a medida que se asignen recursos.
- Vector de recursos disponibles (A): A medida que el sistema va dando recursos a los diferentes procesos, el sistema puede ir guardando en otro vector cuantos hay disponibles. Este vector tendrá tantas entradas como recursos, tal y como el vector de recursos existentes (E), pero donde en cada posición se guardará el número de instancias de cada recurso que se encuentran disponibles, a la espera de ser asignadas a un proceso. Si en un determinado momento el número de una posición es 0, lo que indica es que todas las instancias de ese recurso ya han sido asignadas. Consecuentemente, este vector irá cambiando durante la ejecución del sistema, modificando sus valores a medida que se van asignando y liberando recursos. Además, hay que tener en cuenta que para todas las posiciones del vector A, estos valores serán menores o iguales que los del vector E.
- Matriz de asignaciones actuales (C): Esta matriz se encarga de guardar la información sobre qué recursos han sido asignados a cada proceso. Esta matriz tiene tantas filas como procesos, y tantas columnas como recursos. En esta matriz, se guarda la cantidad de instancias de cada recurso que ya han sido asignadas a cada proceso. Guardar también la situación de cada proceso. Se guarda en una matriz. Tantas columnas como recursos y tantas filas como procesos.

Mediante estos tres conceptos, se define la fórmula:

Con estos tres elementos el SO tiene control absoluto de la situación actual respecto a la situación de recursos de todos los procesos. Para poder aplicar el algoritmo de detección de interbloqueo se necesita otra matriz. La matriz de requerimientos

- Matriz de requerimientos (R): Matriz que almacena cuántos recursos de cada tipo todavía queda por pedir a cada proceso. Por ejemplo, R12 nos dice cuántas instancias del recurso 2 todavía le quedan por pedir al proceso número 1. Esta matriz es problemática ya que generalmente no se conoce, ya que es de futuro. Salvo en situaciones donde se conozca esta matriz, no se puede aplicar este algoritmo.

El algoritmo de detección de interbloqueos se basa en tres pasos:

1. Buscar un proceso desmarcado i para el que la i -ésima fila de R sea menor o igual que A.
2. Si existe dicho proceso, agregar la i -ésima fila de C a A, marcar el proceso y volver al paso 1.
3. Si no existe dicho proceso, el algoritmo termina

Ejemplo:

	Tape drives	Plotters	Scanners	CD Roms		Tape drives	Plotters	Scanners	CD Roms	
E = (	4	2	3	1)		A = (	2	1	0	0)
Current allocation matrix						Request matrix				
C =	$\begin{bmatrix} 0 & 0 & 1 & 0 \\ 2 & 0 & 0 & 1 \\ 0 & 1 & 2 & 0 \end{bmatrix}$					$R = \begin{bmatrix} 2 & 0 & 0 & 1 \\ 1 & 0 & 1 & 0 \\ 2 & 1 & 0 & 0 \end{bmatrix}$				

En un momento dado se tiene la situación de la imagen. Se tienen tres procesos y cuatro recursos. Para el proceso 1, tiene 1 instancia del tipo 3, el proceso 2 tiene 2 instancias del recurso 1... En este caso el SO conoce la matriz R de peticiones, por lo que se puede aplicar el algoritmo de detección de interbloqueo.

El algoritmo funciona de la siguiente forma:

Se coge el vector de disponibles y se comprueba si con dicho el vector se puede acabar alguno de los procesos. En la matriz R, el proceso 1 no se puede acabar en esta situación porque se pide del recurso 4, pero no se tiene en la matriz A. Se prueba con el siguiente, con el proceso 2, pero no se puede porque no hay recurso 3. En el tercer proceso si se puede, porque están los recursos que pide en la fila correspondiente de la matriz R. Lo que se hace en este caso es imaginar que se dan esos recursos, sin asignarlo realmente, haciendo cuentas sobre lo que pasaría para comprobar si pueden darse interbloqueos. Si se le dieran los recursos necesarios a este proceso, este eventualmente acabaría y devolvería todos los recursos que se encontraba ocupando, es decir, devolvería una instancia del recurso 2 y dos instancias del recurso 3, además de los que se le asignaron para que finalizase. De esta manera, el vector de disponibles quedaría de la forma (2,2,2,0).

Con este nuevo vector, el cual es hipotético, se repite el algoritmo y se comprueba si se pueden asignar los recursos disponibles a otro proceso para que termine. Estas cuentas se hacen nuevamente y se comprueba que se puede satisfacer al proceso 2. Se darían los recursos, acabaría, y devolvería los recursos que estaba ocupando. El nuevo vector de recursos disponibles sería (4,2,2,1), con el cual se podría satisfacer al proceso 1 y de esta manera finalizar todos los procesos.

Si el sistema termina de esta forma, se puede demostrar que no se producen interbloqueos, ya que todos los procesos pueden finalizar sin quedar bloqueados indefinidamente. Sin embargo, si en alguno de estos pasos hay algún proceso que no se puede acabar, se produce un interbloqueo. Hay que tener en cuenta que, si en un momento dado se pueden acabar 2 procesos, se puede escoger de manera indiferente cualquiera de los 2 sin que afecte a la posibilidad de ocurrencia de interbloqueos. Hay que tener en cuenta que todo este algoritmo se produce de manera teórica, sin asignar los recursos a los bloqueos, de manera que solo sirve para poder saber si se va a producir un interbloqueo en la futura ejecución de los procesos.

La cuestión principal sobre este algoritmo consiste en saber cuándo emplear este algoritmo, ya que parar el sistema para realizar las numerosas cuentas puede repercutir negativamente en el sistema, ocupando mucha CPU. Hay posibles soluciones: Cada vez que un proceso quiera un recurso, detectando la presencia de un interbloqueo lo antes posible; Cuando hay varios procesos que llevan un tiempo considerable bloqueados; Cada cierto tiempo determinado.

3. Recuperación de un interbloqueo

El sistema, una vez detectado el interbloqueo con el algoritmo previamente explicado puede:

1. Puede **expropiar algún recurso a algún proceso**. Esta solución consiste en quitarle algún recurso a uno de los procesos involucrados. Es una solución drástica que puede derivar en que deje de funcionar algún proceso.
2. Otra posibilidad es **matar algún proceso**. Una vez identificado el proceso que no se puede satisfacer con el algoritmo anterior. Es una solución muy drástica también, aunque eventualmente el resto de los procesos podrían seguir funcionando.
3. **Check-pointing**. Consiste en ir guardando el estado de los procesos, cada cierto tiempo, para que si en el futuro detectas un interbloqueo vuelvas a ese momento previamente guardado. Es una solución menos arriesgada que las anteriores, pero requiere mucho tiempo y memoria, ya que tiene que guardar el estado de todos los procesos, y la recuperación del estado anterior es muy costosa.

4. Evitar Interbloqueos

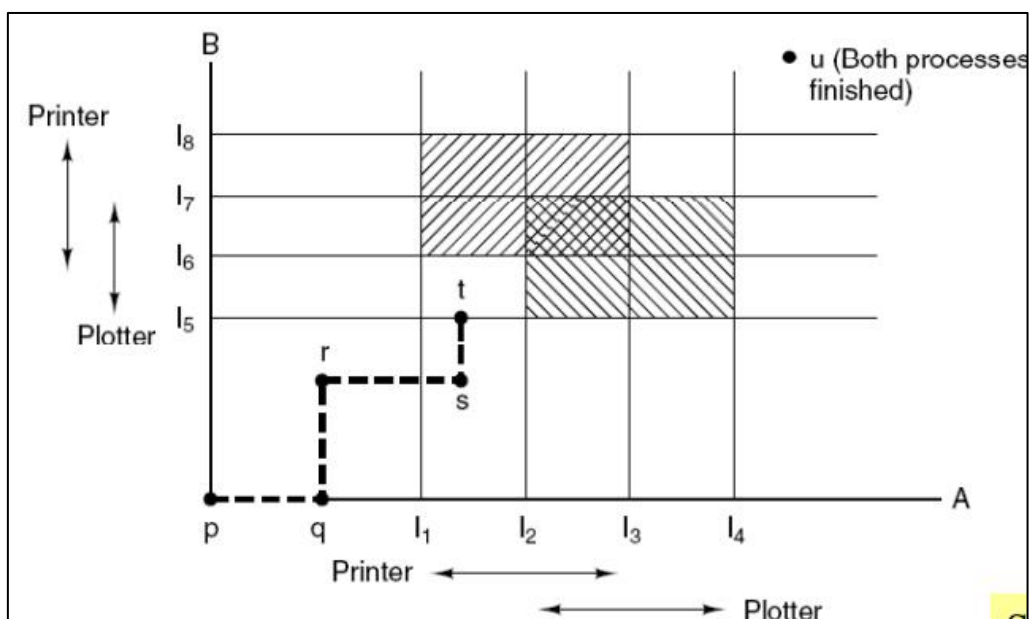


Imagen 3. Trayectorias de los recursos.

La imagen 3 es una figura en la que se muestra gráficamente la existencia de un interbloqueo. En el eje x se representa la evolución de la ejecución del proceso A y en el eje y la ejecución del proceso B. A lo largo de la ejecución de uno de los procesos, por ejemplo, el A, se marcan los momentos en los que se solicita algún recurso, que serían los puntos I1, I2, I3 e I4. Por ejemplo, el punto I1 sería el mutex lock ya que es el momento en el que solicita el recurso "Printer", y el punto I3 sería el mutex unlock ya que es cuando libera ese recurso. Entre los puntos I2 e I3, el proceso A tiene de forma exclusiva los recursos "Printer" y "Plotter".

El proceso B va a querer los mismos recursos que el proceso A pero en un orden diferente. Solicita el recurso, "Plotter", en el punto I5 y lo libera en el I7, y solicita el recurso "Printer" en el punto I6 y lo libera en el I8.

Este es el caso más sencillo de interbloqueo que se puede dar, dos procesos, y dos recursos que se solicitan en diferente orden y de forma exclusiva. Centrándonos en el recurso Printer podemos ver que el acceso del proceso A y del proceso B a ese recurso, coinciden en el rectángulo resaltado en amarillo que se puede ver en la siguiente imagen. El acceso del proceso A y el del proceso B al recurso Plotter se puede ver en el rectángulo rojo de la imagen.

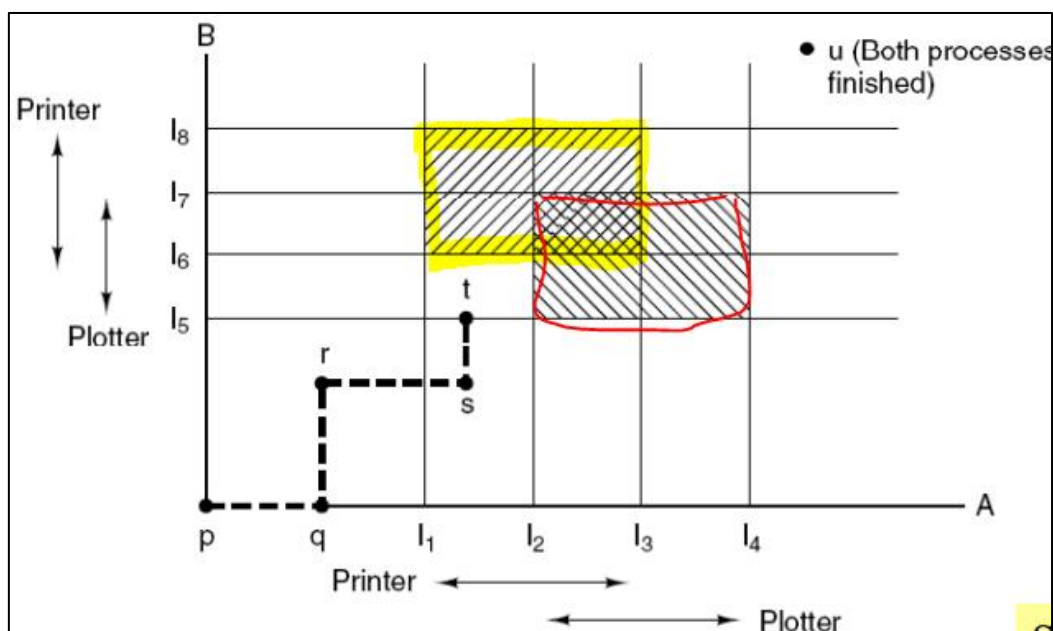


Imagen 4. Trayectorias de los recursos II.

Entrar en esa zona coloreada en amarillo implicaría que tanto el proceso A como el proceso B, accederían a la vez al mismo recurso. Esto es imposible ya que el acceso a los recursos es exclusivo por parte de un único proceso. Esto deriva en que esa parte de la gráfica no va a ser accesible, ya que implicaría algo que no es posible. Al hacer lo propio con el recurso plotter obtenemos una segunda zona coloreada en rojo, a la que tampoco se puede entrar por la misma razón anterior. Si la trayectoria de los procesos no pasa por las zonas coloreadas, la ejecución va a tener éxito y se va a completar. En el camino que está dibujado en la imagen, se produce una situación en la que independientemente de la trayectoria que tome el camino, se va a entrar en alguna de las zonas en las que forma el interbloqueo, por lo que la ejecución va a quedar bloqueada sin posibilidad de salir del interbloqueo.



Grado en Ingeniería Informática

2º CURSO – 2º CUATRIMESTRE

SISTEMAS OPERATIVOS II

06-05-2021

Autores:



Clase 12

Temas trabajados:

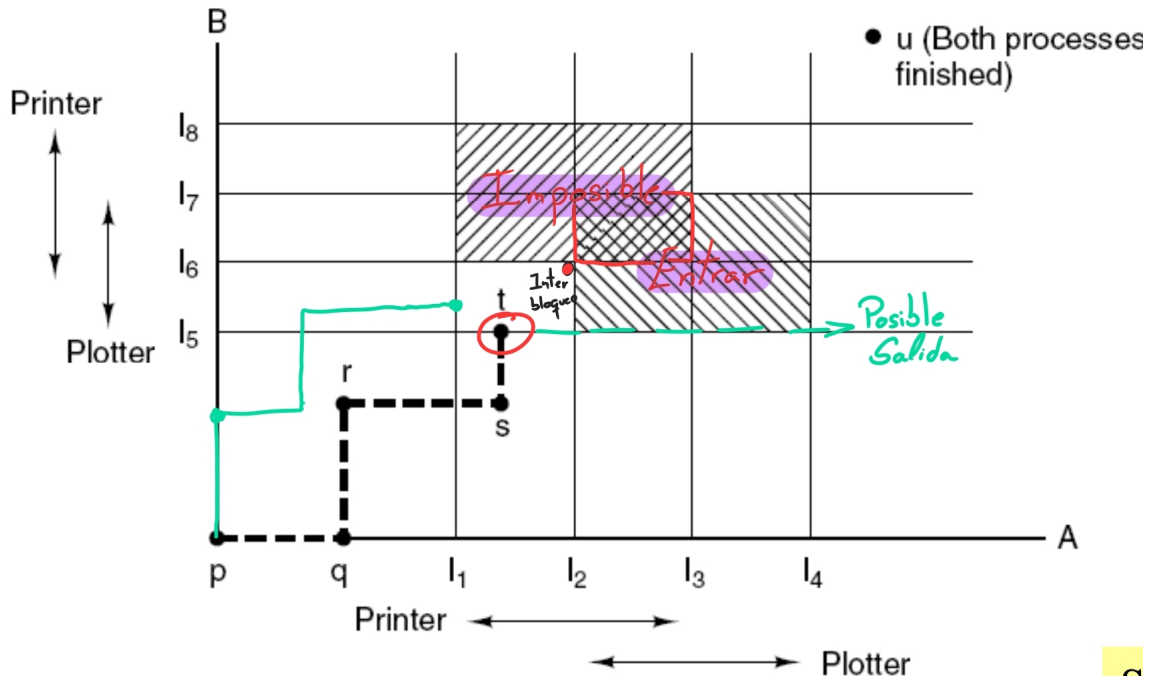
Tema 2: páginas 18–23

Índice

1. Evitar Interbloqueos	2
1.1. Trayectoria de los recursos	2
1.2. Estados seguros e inseguros	3
1.3. El algoritmo del banquero para un solo recurso	3
1.4. El algoritmo del banquero para varios recursos	4
2. Prevención de interbloqueos	4
3. Otras cuestiones	4

1. Evitar Interbloqueos

1.1. Trayectoria de los recursos



Representación del paso del tiempo en la ejecución combinada de los procesos A y B de forma escalonada. La forma del escalón depende de las decisiones que tome el planificador cuando ocurran los cambios de contexto. Las zonas sombreadas representan cuando los dos procesos comparten el uso de un mismo recurso.

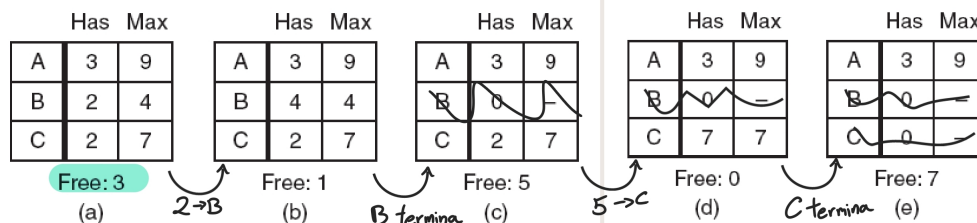
Cuando se está en una situación como la que representa la línea de color negro, cuando se están en el instante de tiempo marcado por el círculo rojo, si se le cede el recurso I_5 al proceso B, se llegaría a un interbloqueo (punto rojo) que provocaría que ninguno de los dos procesos siga con su ejecución. El sistema operativo no puede permitir que esto suceda, por lo que el planificador lo que hará será provocar que la ejecución, de forma gráfica, hacia la derecha (línea verde), cediéndole los recursos a A.

Otra evolución posible es la descrita con la línea verde de la parte derecha. Al llegar al punto final, nos encontramos en la misma situación que la descrita anteriormente, donde en este caso, el planificador debe dejar de darle los recursos a A y solo podrá dárselos a B para evitar el interbloqueo.

Esta gráfica solo se puede realizar si conocemos cual es la traza de ejecución de los procesos y si sabemos en que orden se solicitan y liberan los recursos (conocer el futuro), por lo que no es aplicable para un sistema de propósito general.

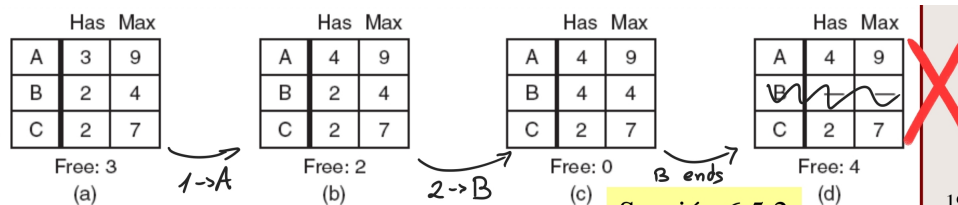
1.2. Estados seguros e inseguros

- Un estado se denomina seguro si hay un cierto orden de programación en el que se puede ejecutar cada proceso hasta completarlo.
- Demostración de que el estado en (a) es seguro:



Se van cediendo los recursos a determinados procesos para conseguir su valor Max y así hacer que el proceso termine. En este caso se ceden los recursos de tal manera que todos los procesos terminan.

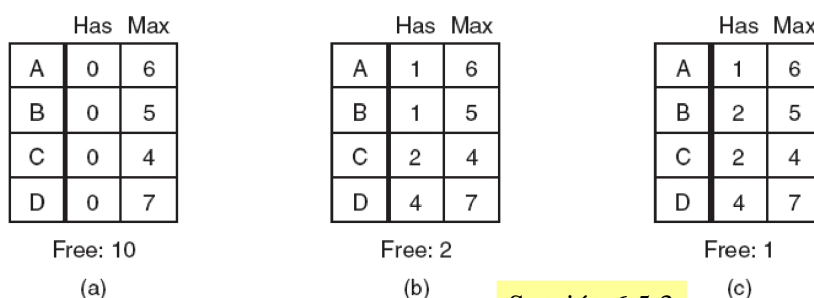
- Demostración de que el estado (b) no es seguro.



En este caso la forma en la que se ceden los recursos provoca que ni el estado A ni C pueden acabar su ejecución por lo que se trata de un estado no seguro.

1.3. El algoritmo del banquero para un solo recurso

- Se buscan procesos que puedan ser satisfechos, y se determina si los recursos liberados cuando acabe pueden satisfacer a otro de manera recursiva.
- (a) y (b) son estados seguros, (c) es un estado inseguro.



Si nos situamos en la figura (b) y el proceso B pide un recurso, aplicando el algoritmo del banquero, podemos comprobar que se llega a un estado (c) no seguro por lo que el proceso B es bloqueado. Realmente no sabemos si al llegar al estado (c) otro proceso podría liberar recursos y por lo tanto no producirse un interbloqueo, sin embargo, este algoritmo bloquea igualmente al proceso.

1.4. El algoritmo del banquero para varios recursos

	Process	Tape drives	Plotters	Printers	CD ROMs
A	3	0	1	1	
B	0	1	0	0	
C	1	1	1	0	
D	1	1	0	1	
E	0	0	0	0	
Resources assigned					

	Process	Tape drives	Plotters	Printers	CD ROMs
A	1	1	0	0	
B	0	1	1	2	
C	3	1	0	0	
D	0	0	1	0	
E	2	1	1	0	
Resources still needed					

E = (6342) Disponibles

P = (5322) Asignados

A = (1020) Sin Asignar

- Dada esta situación si un proceso pide un recurso, se evalúa el estado actual, el estado al que se llegaría si se le cede el recurso y se valora si se puede dar o no en función de si el estado es seguro o inseguro aplicando el algoritmo anterior.
- **IMPORTANTE:** El algoritmo del banquero trabaja petición a petición de recurso, ya que la función mutex_lock es atómica y los recursos son pedidos de uno en uno.

2. Prevención de interbloqueos

- Atacando la condición de exclusión mutua
El demonio de impresión es el único en acceder a la impresora
- Atacando la condición de contención y espera
Los procesos solicitan todos los recursos al principio
Para adquirir un nuevo recurso se liberan todos los que tiene
- Atacando la condición no apropiativa
Virtualización
- Atacando la condición de espera circular
Ordenación de los recursos

Todas estas soluciones tienen grandes defectos y hacen que no sean asumibles en situaciones de propósito general.

3. Otras cuestiones

- Bloqueo de dos fases: Se piden todos los recursos a la vez y si no se le conceden libera todos. Pasando un tiempo vuelve a pedirlos. Un proceso solo puede tener o todos los recursos o ninguno.

- Interbloqueos de comunicaciones: Relacionado con Redes, interbloqueos con los agradecimientos (ACK). Se soluciona reenviando mensajes si no se reciben los agradecimientos y comprobando si se reciben mensajes repetidos.
- Bloqueo activo (livelock): **Problema muy grave:** En el acceso a un recurso el programador ha empleado la espera activa. Esto, no es un gran problema para las carrera críticas, si es un problema para el rendimiento y sobre todo es un gran problema para los interbloqueos ya que es posible que dos procesos se queden en espera activa porque están en un interbloqueo. Este interbloqueo pasa desapercibido ya que los procesos se siguen ejecutando (bucle de la espera activa).
- Inanición: Provocada por la inversión de prioridad. Se tiene un sistema en el que un proceso prioritario esta esperando en un bloqueo activo que se libere un recurso, sin embargo, este recurso lo tiene un proceso de menor prioridad. Como el planificador intenta dar los recursos a los procesos mas importantes, el primer proceso nunca recibirá el recurso que tiene el proceso menos prioritario.